

Title Page

Lab 3 — SYN Flood Pattern Investigation Using TShark

Student Name: Athanasius Orikeze Alekwe

Student ID: 2025/FWSD/11230

**Course Name: Basic Networking Skills for Digital
Forensics**

Instructor Name: Aminu Idris

Date of Submission: 11/09/2026

INTRODUCTION

This practical investigation focused on identifying and analysing unusual TCP SYN activity directed at a local Apache web service. The exercise was designed to show the difference between a normal TCP connection and repeated connection attempts that do not complete the expected three-way handshake.

A normal HTTP connection was first captured to establish a baseline. This provided a clear example of the expected SYN → SYN-ACK → ACK sequence before normal HTTP communication began. A separate controlled test was then performed using Scapy, where only four SYN packets were sent to the local Apache service on 127.0.0.1:80.

TShark was used throughout the investigation to isolate TCP flags, identify the source ports involved, count packets, examine reset behaviour, compare normal and incomplete sessions, and build a packet-level forensic timeline. All traffic generation remained within the authorised loopback environment and the original evidence was preserved before analysis, as required by the lab instructions.

OBJECTIVES

The objectives of this practical were to:

Establish a normal TCP handshake and identify the SYN, SYN-ACK and ACK sequence.

Capture and examine the authorised four-packet SYN simulation.

Use TShark filters to identify initial SYN packets, SYN-ACK responses and ACK/RST behaviour.

Determine the number of SYN packets and unique source ports involved.

Compare a successfully completed TCP session with incomplete or aborted connection attempts.

Construct a timeline showing the sequence of events in the bounded capture.

Explain what the observed packet pattern proves and what cannot be concluded from it.

Preserve the original evidence and verify its integrity using SHA-256 hashing.

Recommend appropriate detection and mitigation measures for suspicious SYN activity.

LABORATORY / TOOLS USED

The practical was completed inside a Kali Linux virtual machine running in VMware Workstation. The Apache service was hosted locally and was confirmed to be listening on TCP port 80 before packet generation began.

The tools used were:

Tool	Version / Purpose
Kali Linux	Investigation environment
Apache HTTP Server	2.4.68 — local web service
TShark	4.4.7 — packet capture and command-line analysis
Wireshark	4.4.7 — packet-analysis support
Scapy	2.6.1 — authorised four-SYN simulation
curl	Generated the legitimate HTTP baseline request
SHA-256	Verified evidence integrity
VMware Workstation	Hosted the isolated Kali environment

EXECUTIVE SUMMARY AND SCOPE

This investigation examined repeated TCP SYN activity against a local Apache service and compared it with a legitimate TCP/HTTP connection.

The normal baseline successfully completed the TCP three-way handshake. Frame 1 contained the client SYN, Frame 2 contained Apache's SYN-ACK, and Frame 3 completed the handshake with the client's ACK. An HTTP GET request followed in Frame 4, and Apache returned an HTTP/1.1 200 OK response in Frame 6.

The controlled SYN simulation behaved differently. Four initial SYN packets were sent to 127.0.0.1:80, using four separate source ports. Apache responded to all four with SYN-ACK packets, but each response was followed by an RST instead of the normal final ACK. Consequently, none of the four simulated connections completed the TCP handshake and no HTTP request followed.

The practical was restricted to the local loopback interface and exactly four SYN packets. Therefore, the findings demonstrate an incomplete-handshake pattern but do not establish that a real denial-of-service attack occurred. The manual specifically warns that a genuine SYN flood must be judged using factors such as scale, rate, persistence and service impact.

EVIDENCE ACQUISITION AND INTEGRITY HASHES

The lab directory was organised into separate folders for original evidence, working copies, exported material, reports, screenshots and scripts. This allowed the original capture files to be preserved separately from files used during analysis.

The supplied training capture was downloaded and stored as:

evidence/mySYNFloodCapture.pcap

Its SHA-256 value was:

14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5

The normal HTTP baseline was stored as:

evidence/normal_http.pcapng

Its SHA-256 value was:

e63a02397ea9293ecf6f3c72fb1b28e5bcc5951ff847d5978e9afcdbfd6a0b

The original bounded simulation capture was stored as:

evidence/bounded_syn_activity.pcapng

A forensic working copy was created as:

working/bounded_syn_activity_working.pcapng

Both bounded files produced the same SHA-256 value:

1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa

The matching values showed that the working copy was identical to the preserved capture when the integrity check was performed. Final verification using sha256sum -c also returned OK for the evidence files. This followed the manual's requirement to preserve originals and analyse verified working copies.

NORMAL TCP HANDSHAKE BASELINE

A legitimate connection to the local Apache server was generated using curl. TShark recorded ten packets in the baseline capture.

The first three frames clearly showed the normal TCP three-way handshake:

Frame	Communication	Result
1	35656 → 80	SYN
2	80 → 35656	SYN-ACK
3	35656 → 80	ACK

After the handshake was completed, Frame 4 contained an HTTP GET request and Frame 6 contained the Apache HTTP/1.1 200 OK response.

This established the expected behaviour of a legitimate connection: the server's SYN-ACK was followed by a final ACK from the client, after which application-layer traffic was exchanged. The session later closed normally using FIN/ACK packets. This baseline was important because it provided a direct comparison for the bounded SYN activity.

BOUNDED SYN SIMULATION

The Scapy script used in the practical was deliberately restricted to:

TARGET = 127.0.0.1

PORT = 80

COUNT = 4

The script was checked before execution to make sure it contained the correct target, port and packet count. It was then executed once while TShark captured traffic on the loopback interface.

The resulting capture contained 12 packets and occupied 1,428 bytes. The relevant packet activity lasted approximately 0.004870879 seconds.

No further SYN packets were generated after the authorised four-packet test. This remained within the manual's requirement that the simulation must contain only four SYN packets and must target localhost or another approved private lab system.

SYN, SYN-ACK AND ACK/RST ANALYSIS

TShark was used to isolate initial SYN packets with:

```
tcp.flags.syn==1 && tcp.flags.ack==0
```

Four initial SYN packets were identified:

Frame	Source Port	Destination
1	26039	127.0.0.1:80
4	54264	127.0.0.1:80
7	26907	127.0.0.1:80
10	28596	127.0.0.1:80

The SYN-ACK filter identified four server responses:

Frame	Source	Destination Port
2	127.0.0.1:80	26039
5	127.0.0.1:80	54264
8	127.0.0.1:80	26907
11	127.0.0.1:80	28596

RST analysis then identified Frames 3, 6, 9 and 12 as reset packets.

The four sequences were therefore:

SYN → SYN-ACK → RST

SYN → SYN-ACK → RST

SYN → SYN-ACK → RST

SYN → SYN-ACK → RST

None of the four connections received the normal final ACK needed to complete the three-way handshake.

QUANTITATIVE ANALYSIS

The bounded capture produced the following results:

Indicator	Result
Initial SYN packets	4
SYN-ACK responses	4
RST packets	4
Completed TCP handshakes	0
HTTP requests	0
Unique client source ports	4
Destination port	80
Activity duration	0.004870879 seconds

The unique source ports were:

26039, 26907, 28596, 54264

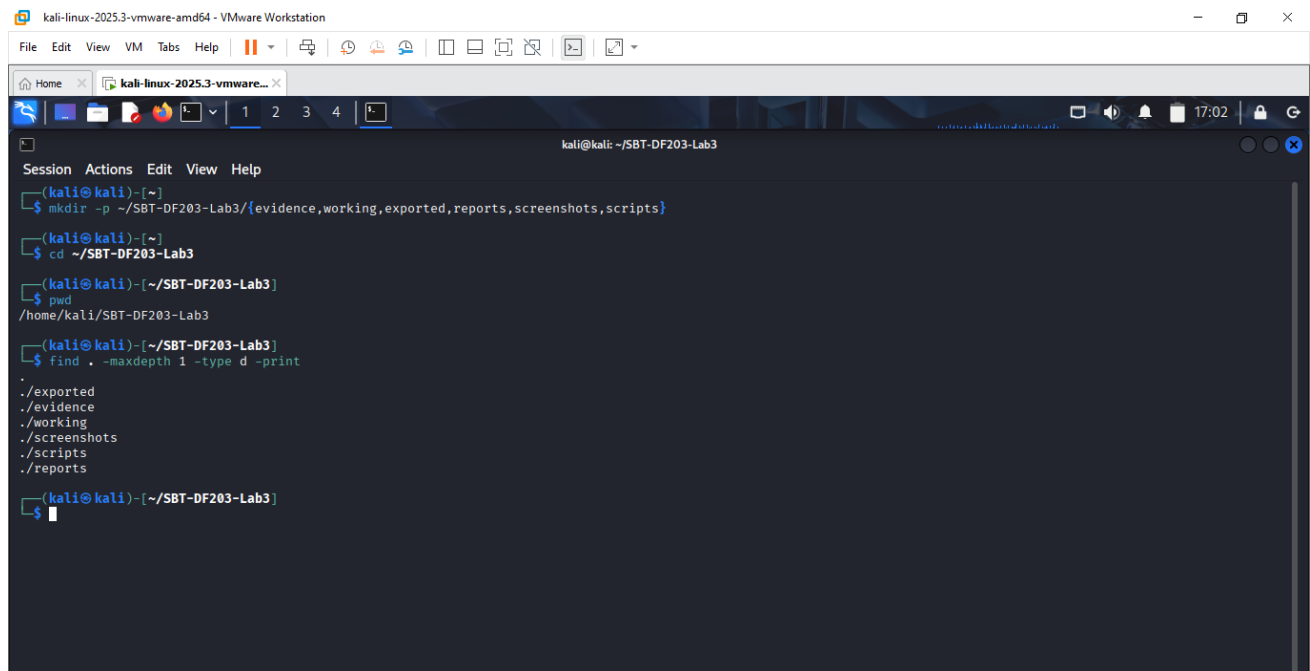
All four initial SYN packets originated from 127.0.0.1, were directed to 127.0.0.1, and targeted TCP port 80.

TShark Expert Information also reported four connection resets, four SYN connection requests and four SYN-ACK responses. The separate TCP retransmission/lost-segment/duplicate-ACK check returned zero matching events.

These results quantify the repeated incomplete connection behaviour rather than relying only on visual inspection of the capture.

Practical Evidence

Create the forensic lab folders



The screenshot shows a terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal prompt is "kali@kali: ~/SBT-DF203-Lab3". The user has executed the following commands:

```
mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab3
pwd
find . -maxdepth 1 -type d -print
```

The output of the 'find' command shows the following directory structure:

```
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

Fig 1.0 Lab folder structure and start time.

Evidence field	Entry
Case/Lab Identifier	SBT-DF203-Lab3
Trainee Name	Athanasius Alekwe
Registration Number	2025/FWSD/11230
Time Started:	2026-09-11

Explanation

This screenshot shows the folder structure created at the beginning of the investigation. Separating evidence, working files, reports, screenshots and scripts helped keep the practical organised and protected the original evidence from unnecessary modification.

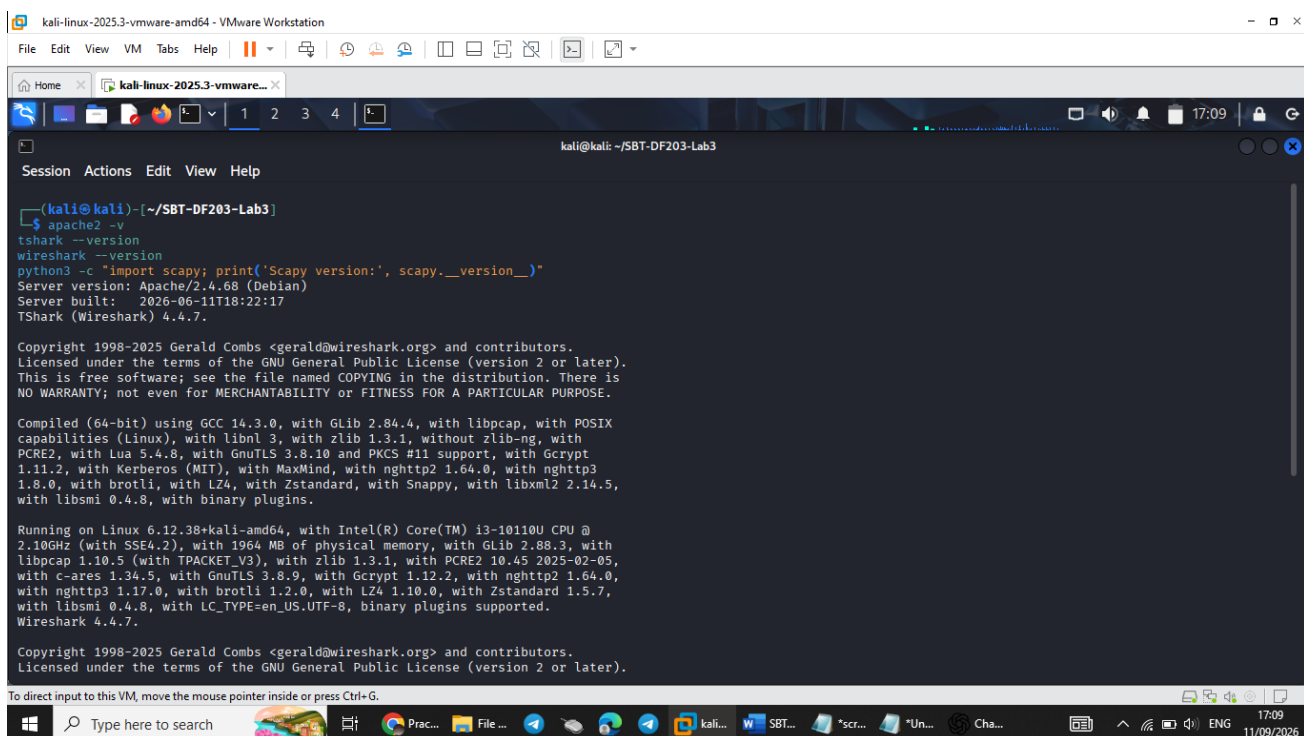


Fig 1.1 Verifying Apache, TShark, Wireshark, Scapy.

Explanation

This screenshot confirms that the main tools required for the practical were installed on the Kali Linux system. It also records the versions used during the investigation, providing useful information about the laboratory environment.

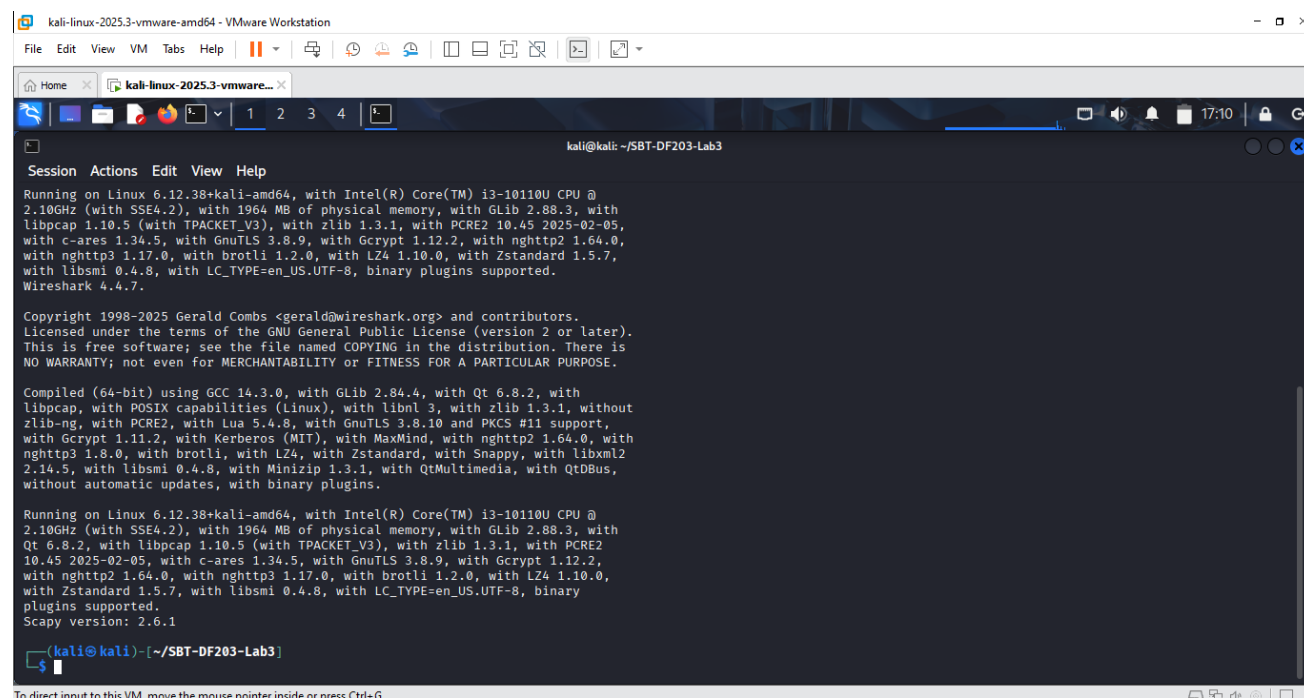
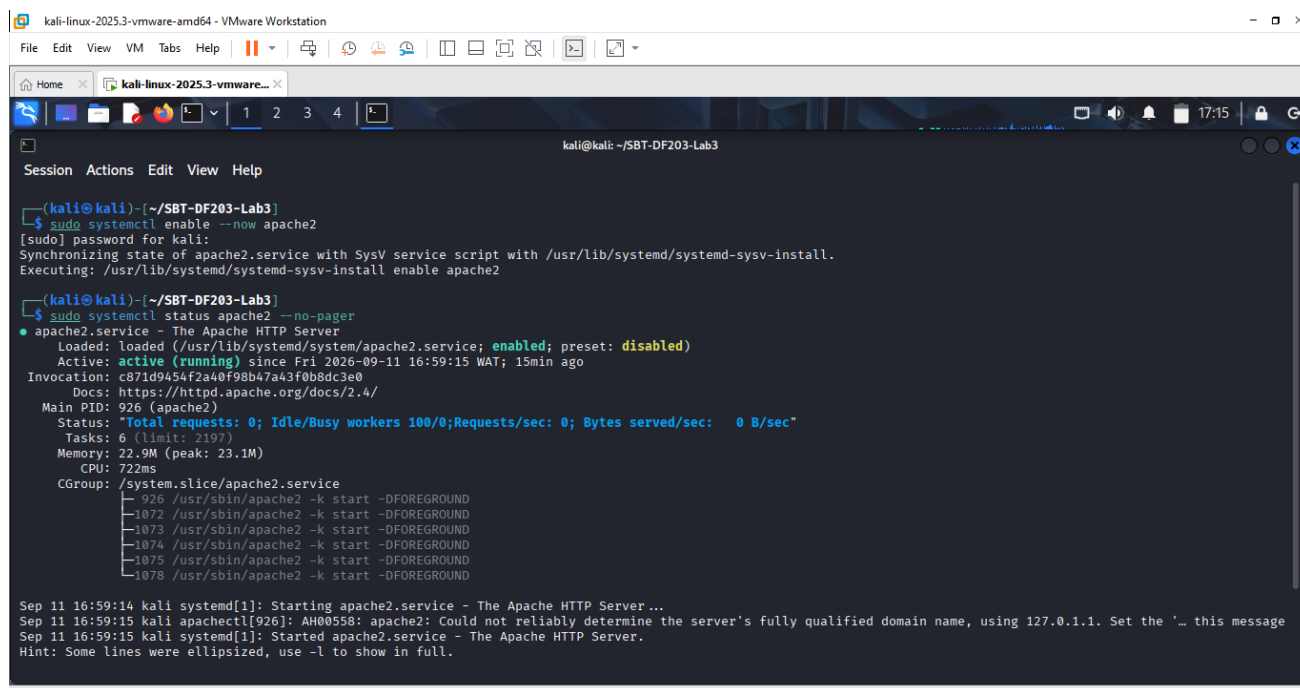


Fig 1.2 Fig 1.1 Verifying Apache, TShark, Wireshark, Scapy page 1.

Explanation

This continuation confirms the remaining version information for TShark, Wireshark and Scapy. Together with the previous screenshot, it demonstrates that the required software was available before the practical analysis began.



```
kali@kali: ~/SBT-DF203-Lab3
$ sudo systemctl enable --now apache2
[sudo] password for kali:
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

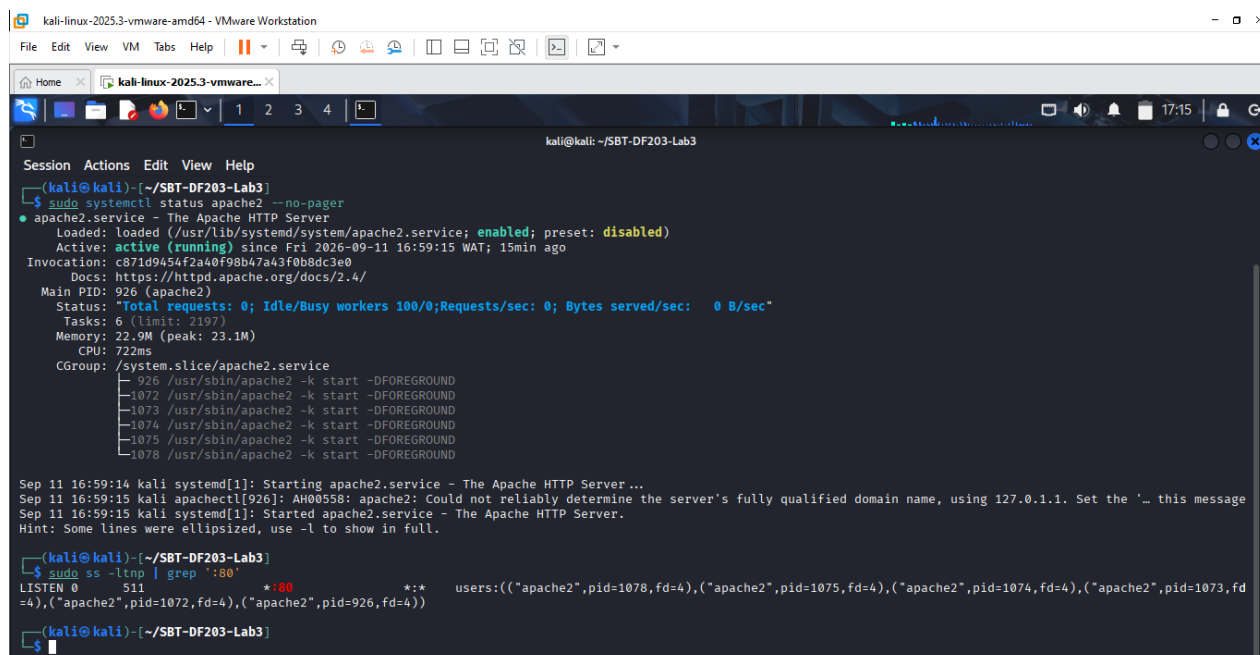
(kali@kali)~[~/SBT-DF203-Lab3]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-11 16:59:15 WAT; 15min ago
     Invocation: c871d9454f2a40f98b47a43f0b8dc3e0
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 926 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 2197)
      Memory: 22.9M (peak: 23.1M)
         CPU: 722ms
    CGroup: /system.slice/apache2.service
            └─ 926 /usr/sbin/apache2 -k start -DFOREGROUND
               1072 /usr/sbin/apache2 -k start -DFOREGROUND
               1073 /usr/sbin/apache2 -k start -DFOREGROUND
               1074 /usr/sbin/apache2 -k start -DFOREGROUND
               1075 /usr/sbin/apache2 -k start -DFOREGROUND
               1078 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 11 16:59:14 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 11 16:59:15 kali apachectl[926]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the '...' this message
Sep 11 16:59:15 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
```

Fig 1.3 Starting Apache page and Confirming port 80.

Explanation

This screenshot shows Apache being enabled and started successfully on the Kali virtual machine. The service status confirms that Apache was active and ready to receive the test connections.



```
kali@kali: ~/SBT-DF203-Lab3
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-11 16:59:15 WAT; 15min ago
     Invocation: c871d9454f2a40f98b47a43f0b8dc3e0
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 926 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 2197)
      Memory: 22.9M (peak: 23.1M)
         CPU: 722ms
    CGroup: /system.slice/apache2.service
            └─ 926 /usr/sbin/apache2 -k start -DFOREGROUND
               1072 /usr/sbin/apache2 -k start -DFOREGROUND
               1073 /usr/sbin/apache2 -k start -DFOREGROUND
               1074 /usr/sbin/apache2 -k start -DFOREGROUND
               1075 /usr/sbin/apache2 -k start -DFOREGROUND
               1078 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 11 16:59:14 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 11 16:59:15 kali apachectl[926]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the '...' this message
Sep 11 16:59:15 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

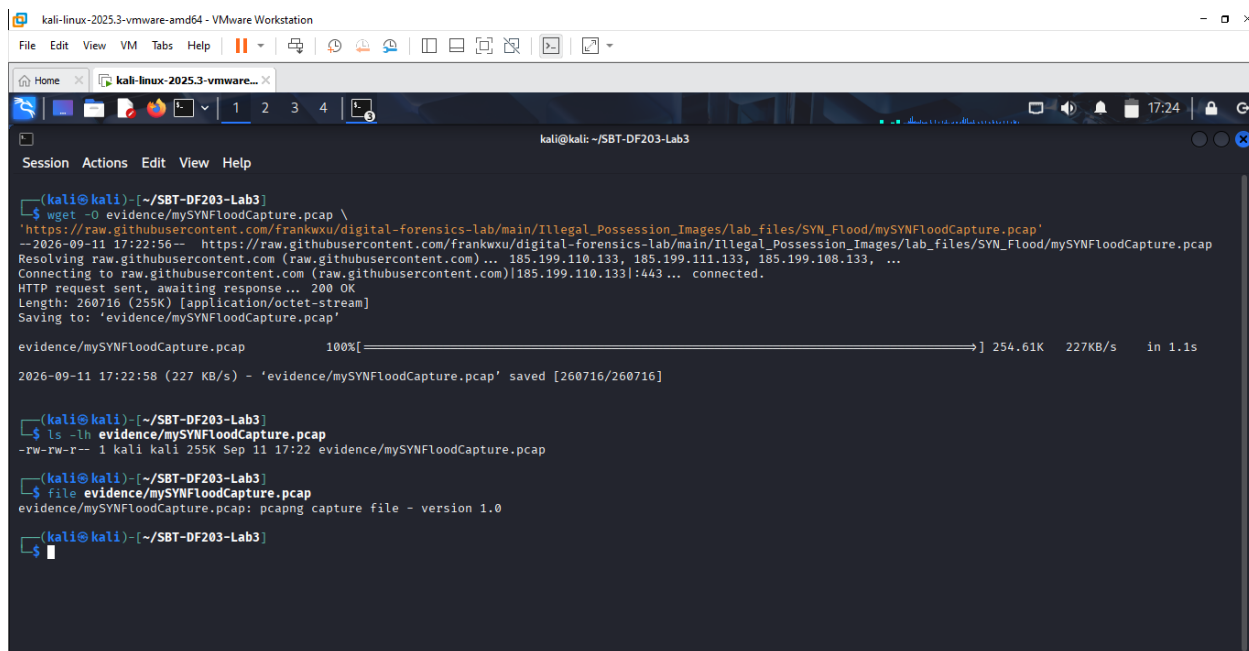
(kali@kali)~[~/SBT-DF203-Lab3]
$ sudo ss -ltnp | grep :80
LISTEN 0      511      *:*        users:((("apache2",pid=1078,fd=4),("apache2",pid=1075,fd=4),("apache2",pid=1074,fd=4),("apache2",pid=1073,fd=4),("apache2",pid=1072,fd=4),("apache2",pid=926,fd=4)))

(kali@kali)~[~/SBT-DF203-Lab3]
$
```

Fig 1.4 Starting Apache page and Confirming port 80 page 1.

Explanation

This screenshot confirms that the Apache service was both active and listening on TCP port 80. This verification was necessary before creating the normal baseline or sending the controlled SYN packets.



```
kali@kali: ~/SBT-DF203-Lab3
$ wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'
--2026-09-11 17:22:56-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.111.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 260716 (255K) [application/octet-stream]
Saving to: 'evidence/mySYNFloodCapture.pcap'

evidence/mySYNFloodCapture.pcap 100%[=====] 254.61K 227KB/s in 1.1s

2026-09-11 17:22:58 (227 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]

(kali@kali) ~/SBT-DF203-Lab3
$ ls -lh evidence/mySYNFloodCapture.pcap
-rw-rw-r-- 1 kali kali 255K Sep 11 17:22 evidence/mySYNFloodCapture.pcap

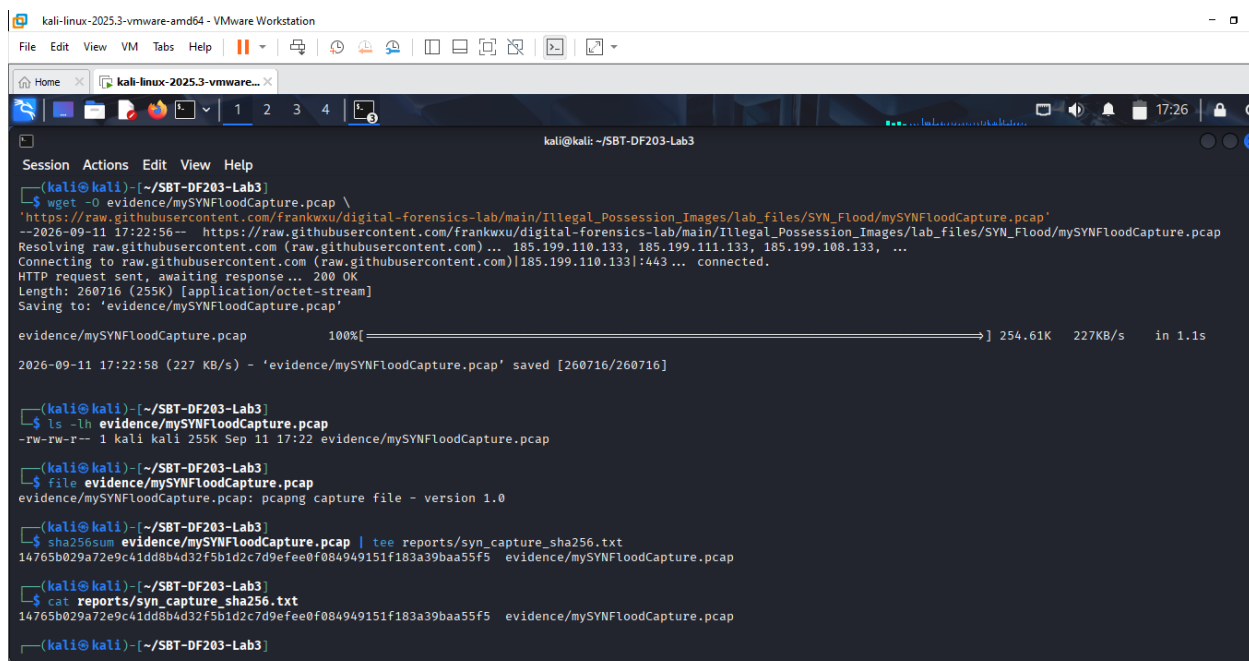
(kali@kali) ~/SBT-DF203-Lab3
$ file evidence/mySYNFloodCapture.pcap
evidence/mySYNFloodCapture.pcap: pcapng capture file - version 1.0

(kali@kali) ~/SBT-DF203-Lab3
$
```

Fig 1.5 Downloading the PCAP and verifying the PCAP file.

Explanation

This screenshot records the download of the supplied SYN-flood training capture into the evidence folder. The file checks confirm that the downloaded evidence was present and recognised as packet-capture data.



```
kali@kali: ~/SBT-DF203-Lab3
$ wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'
--2026-09-11 17:22:56-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.111.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 260716 (255K) [application/octet-stream]
Saving to: 'evidence/mySYNFloodCapture.pcap'

evidence/mySYNFloodCapture.pcap 100%[=====] 254.61K 227KB/s in 1.1s

2026-09-11 17:22:58 (227 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]

(kali@kali) ~/SBT-DF203-Lab3
$ ls -lh evidence/mySYNFloodCapture.pcap
-rw-rw-r-- 1 kali kali 255K Sep 11 17:22 evidence/mySYNFloodCapture.pcap

(kali@kali) ~/SBT-DF203-Lab3
$ file evidence/mySYNFloodCapture.pcap
evidence/mySYNFloodCapture.pcap: pcapng capture file - version 1.0

(kali@kali) ~/SBT-DF203-Lab3
$ sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
14765b029a72e9c41db8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap

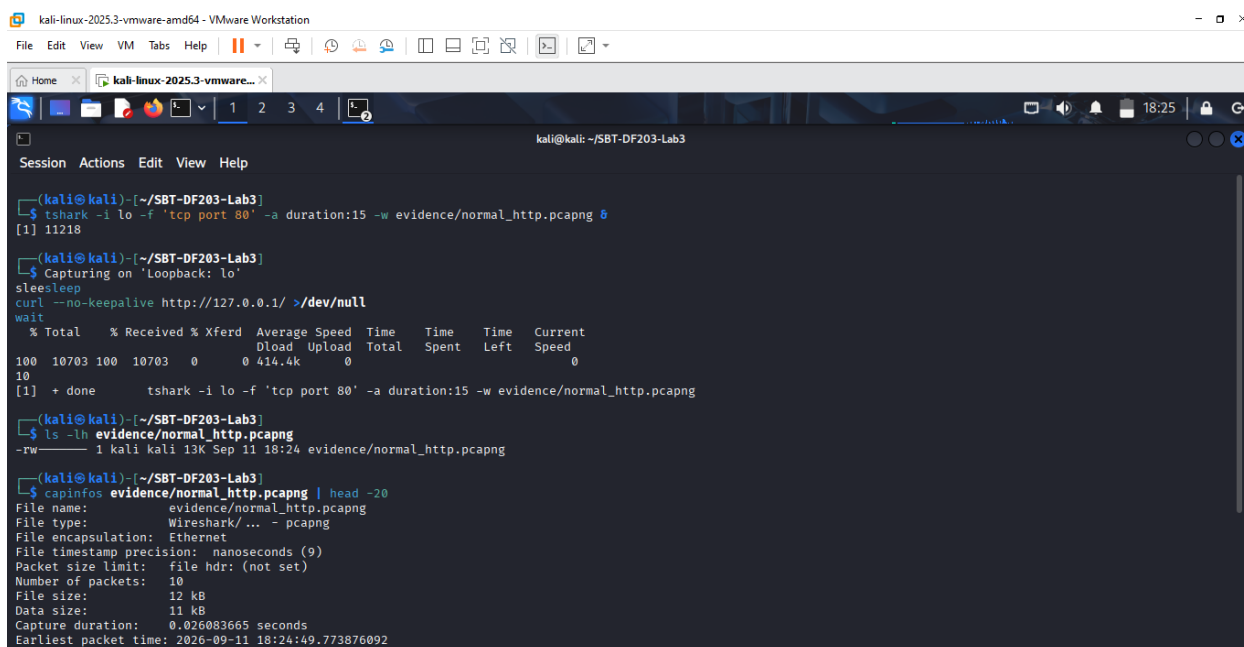
(kali@kali) ~/SBT-DF203-Lab3
$ cat reports/syn_capture_sha256.txt
14765b029a72e9c41db8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap

(kali@kali) ~/SBT-DF203-Lab3
$
```

Fig 1.6 Calculating the original SHA-256 hash and verifying the hash file was created.

Explanation

This screenshot shows the SHA-256 hash calculated for the supplied training capture. Recording the hash created an integrity reference that could later be used to confirm that the evidence had not changed.

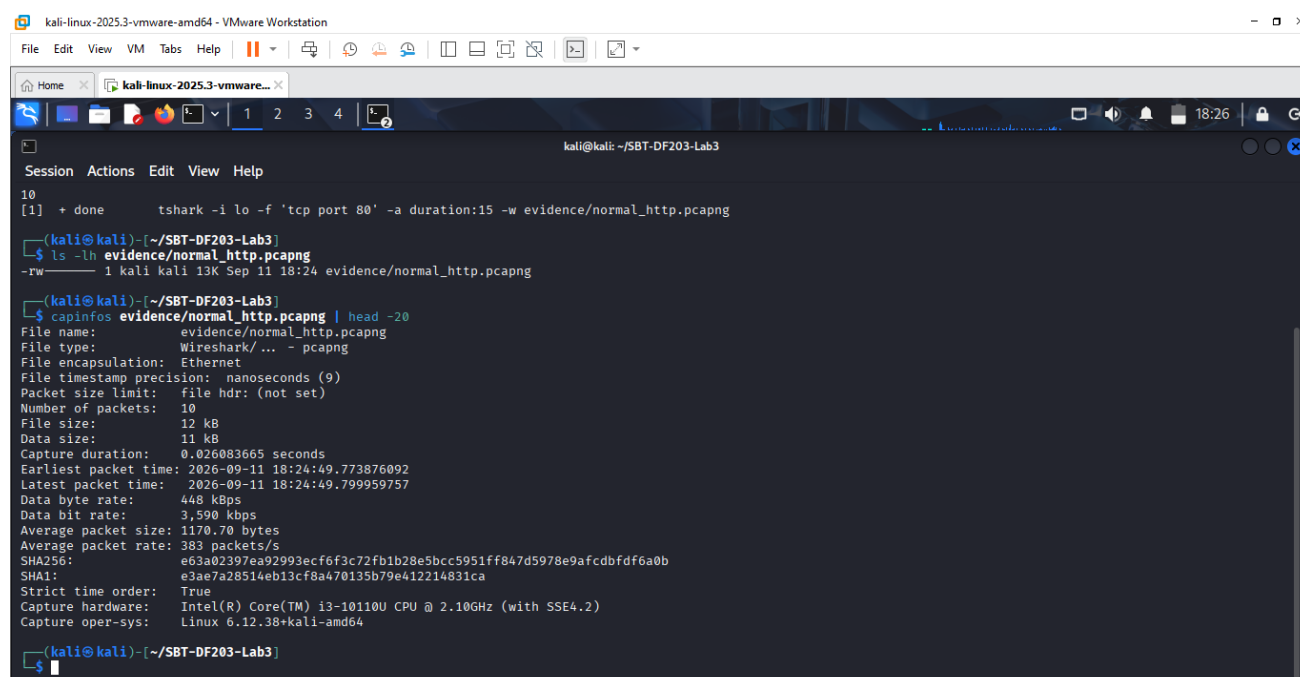


```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
kali@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
kali@kali:~/SBT-DF203-Lab3
$ tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng 6
[1] 11218
kali@kali:~/SBT-DF203-Lab3
$ Capturing on 'Loopback: lo'
sleep
curl --no-keepalive http://127.0.0.1/>/dev/null
wait
% Total % Received % Xferd Average Speed Time Time Time Current
      Dload Upload Total Spent Left Speed
100 10703 100 10703 0 0 414.4k 0 0 0 0
10
[1] + done tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng
kali@kali:~/SBT-DF203-Lab3
$ ls -lh evidence/normal_http.pcapng
-rw-r--r-- 1 kali kali 13K Sep 11 18:24 evidence/normal_http.pcapng
kali@kali:~/SBT-DF203-Lab3
$ capinfos evidence/normal_http.pcapng | head -20
File name: evidence/normal_http.pcapng
File type: Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: nanoseconds (9)
Packet size limit: file hdr: (not set)
Number of packets: 10
File size: 12 kB
Data size: 11 kB
Capture duration: 0.026083665 seconds
Earliest packet time: 2026-09-11 18:24:49.773876092
```

Fig 1.7 Generating one legitimate HTTP request.

Explanation

This screenshot shows TShark capturing traffic on the loopback interface while a normal request was sent to the local Apache server. The purpose was to create a legitimate TCP/HTTP baseline for comparison with the bounded SYN activity.

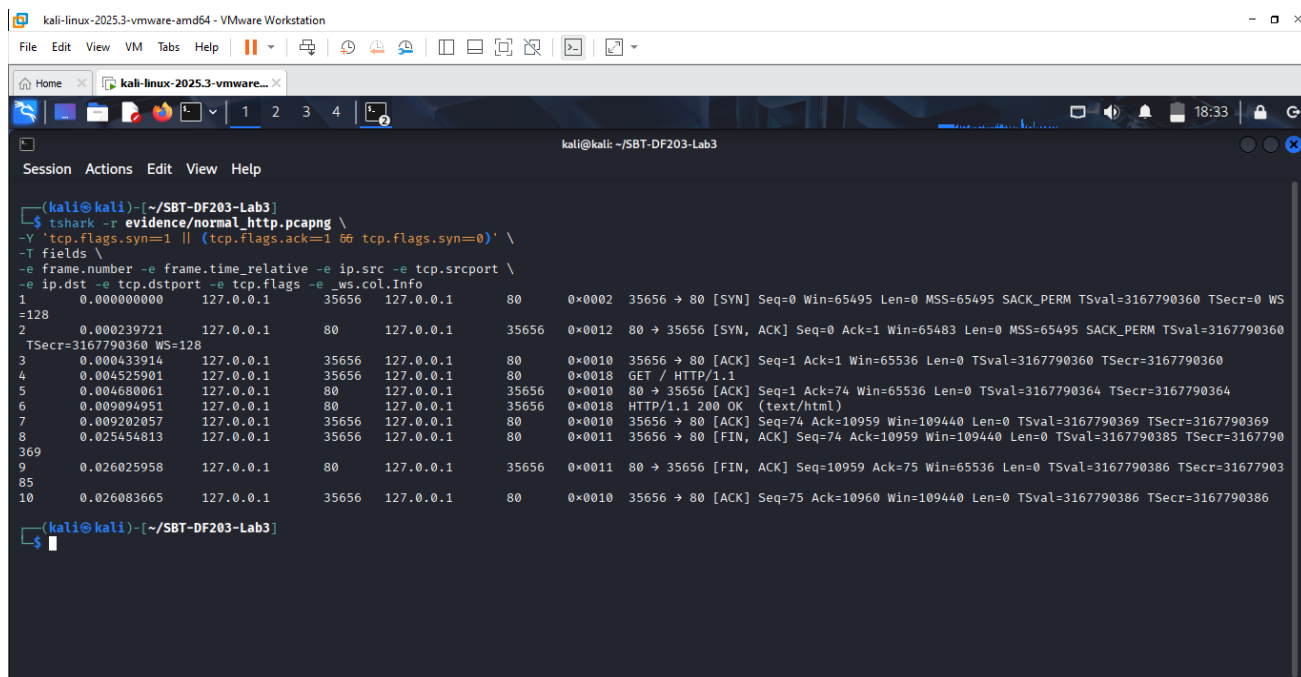


```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
kali@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
10
[1] + done tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng
kali@kali:~/SBT-DF203-Lab3
$ ls -lh evidence/normal_http.pcapng
-rw-r--r-- 1 kali kali 13K Sep 11 18:24 evidence/normal_http.pcapng
kali@kali:~/SBT-DF203-Lab3
$ capinfos evidence/normal_http.pcapng | head -20
File name: evidence/normal_http.pcapng
File type: Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: nanoseconds (9)
Packet size limit: file hdr: (not set)
Number of packets: 10
File size: 12 kB
Data size: 11 kB
Capture duration: 0.026083665 seconds
Earliest packet time: 2026-09-11 18:24:49.773876092
Latest packet time: 2026-09-11 18:24:49.799959757
Data byte rate: 448 kbps
Data bit rate: 3,590 kbps
Average packet size: 1170.70 bytes
Average packet rate: 383 packets/s
SHA256: e63a02397ea92993ecf6f3c72fb1b28e5bcc5951ff847d5978e9afcdcbdf6a0b
SHA1: e3ae7a28514eb13cf8a470135b79e412214831ca
Strict time order: True
Capture hardware: Intel(R) Core(TM) i3-10110U CPU @ 2.10GHz (with SSE4.2)
Capture oper-sys: Linux 6.12.38+kali-amd64
kali@kali:~/SBT-DF203-Lab3
$
```

Fig 1.8 Generating one legitimate HTTP request page 1.

Explanation

This screenshot confirms that normal_http.pcapng was created successfully and contains captured packets. The capture information provided the basic file details and timing information for the legitimate baseline session.

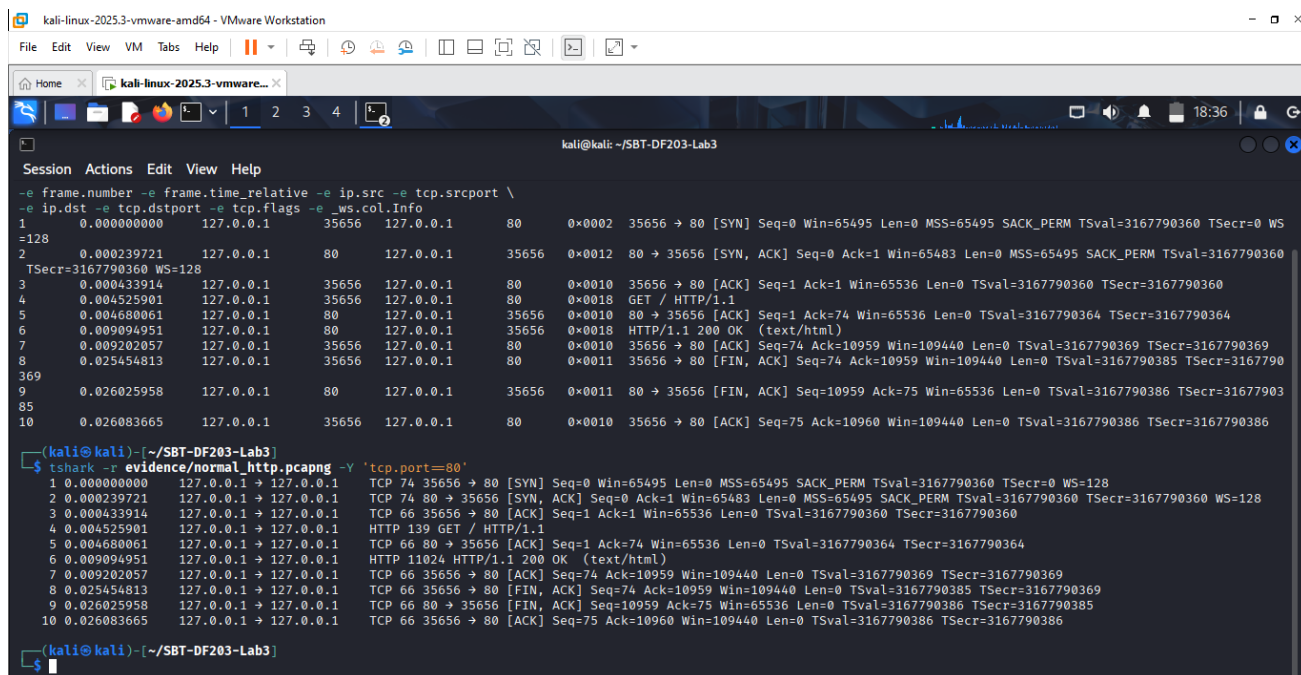


```
kali@kali: ~/SBT-DF203-Lab3
$ tshark -r evidence/normal_http.pcapng \
-Y 'tcp.flags.syn==1 || (tcp.flags.ack==1 && tcp.flags.syn==0)' \
-T fields \
-e frame.number -e frame.time_relative -e ip.src -e tcp.srcport \
-e ip.dst -e tcp.dstport -e tcp.flags -e _ws.col.Info
1 0.000000000 127.0.0.1 35656 127.0.0.1 80 0x0002 35656 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3167790360 TSecr=0 WS=128
2 0.000239721 127.0.0.1 80 127.0.0.1 35656 0x0012 80 → 35656 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=3167790360 TSecr=3167790360 WS=128
3 0.000433914 127.0.0.1 35656 127.0.0.1 80 0x0010 35656 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3167790360 TSecr=3167790360
4 0.004525901 127.0.0.1 35656 127.0.0.1 80 0x0018 GET / HTTP/1.1
5 0.004680061 127.0.0.1 80 127.0.0.1 35656 0x0010 80 → 35656 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=3167790364 TSecr=3167790364
6 0.009094951 127.0.0.1 80 127.0.0.1 35656 0x0018 HTTP/1.1 200 OK (text/html)
7 0.009202057 127.0.0.1 35656 127.0.0.1 80 0x0010 35656 → 80 [ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=3167790369 TSecr=3167790369
8 0.025454813 127.0.0.1 35656 127.0.0.1 80 0x0011 35656 → 80 [FIN, ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=3167790385 TSecr=3167790385
9 0.026025958 127.0.0.1 80 127.0.0.1 35656 0x0011 80 → 35656 [FIN, ACK] Seq=10959 Ack=75 Win=65536 Len=0 TSval=3167790386 TSecr=3167790386
10 0.026083665 127.0.0.1 35656 127.0.0.1 80 0x0010 35656 → 80 [ACK] Seq=75 Ack=10960 Win=109440 Len=0 TSval=3167790386 TSecr=3167790386
```

Fig 1.9 Showing the complete three-way handshake clearly.

Explanation

This screenshot shows the normal connection beginning with SYN, followed by SYN-ACK and the final ACK. It demonstrates the correct three-way TCP handshake expected before application-layer communication starts.



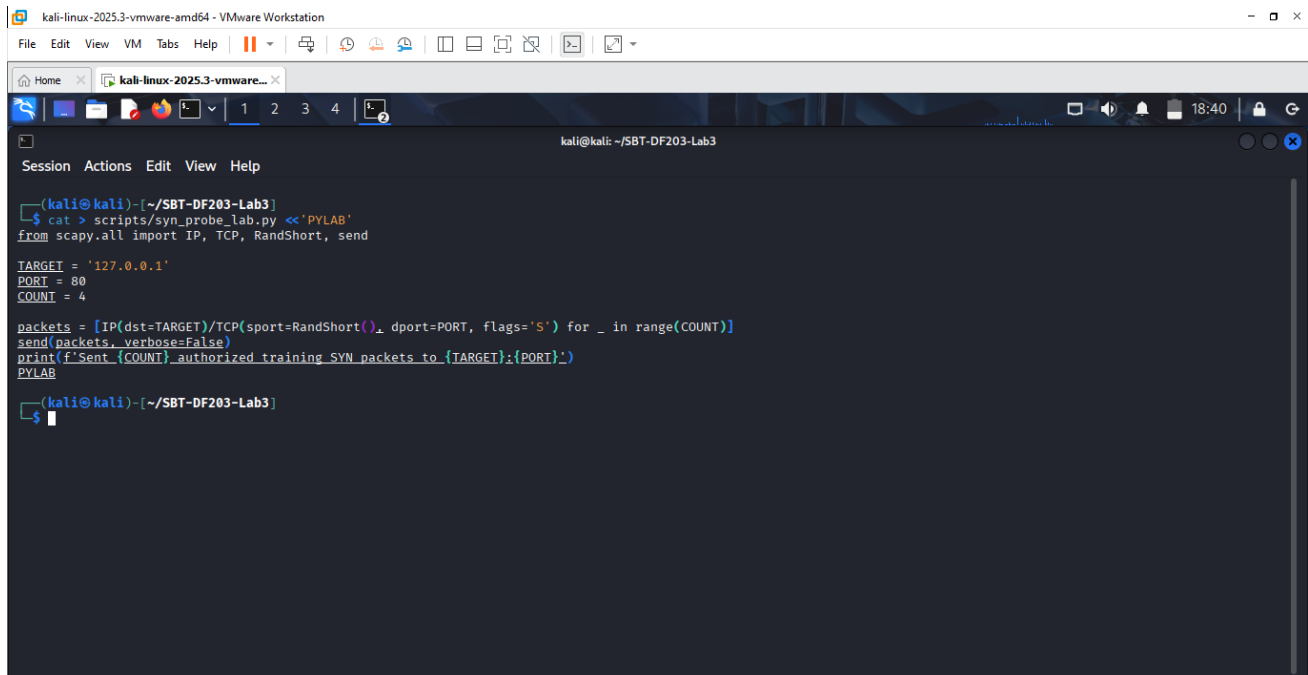
```
kali@kali: ~/SBT-DF203-Lab3
$ tshark -r evidence/normal_http.pcapng -Y 'tcp.port==80'
1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 35656 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3167790360 TSecr=0 WS=128
2 0.000239721 127.0.0.1 → 127.0.0.1 TCP 74 80 → 35656 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=3167790360 TSecr=3167790360 WS=128
3 0.000433914 127.0.0.1 → 127.0.0.1 TCP 66 35656 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3167790360 TSecr=3167790360
4 0.004525901 127.0.0.1 → 127.0.0.1 HTTP 139 GET / HTTP/1.1
5 0.004680061 127.0.0.1 → 127.0.0.1 TCP 66 80 → 35656 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=3167790364 TSecr=3167790364
6 0.009094951 127.0.0.1 → 127.0.0.1 HTTP 11024 HTTP/1.1 200 OK (text/html)
7 0.009202057 127.0.0.1 → 127.0.0.1 TCP 66 35656 → 80 [ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=3167790369 TSecr=3167790369
8 0.025454813 127.0.0.1 → 127.0.0.1 TCP 66 35656 → 80 [FIN, ACK] Seq=74 Ack=10959 Win=109440 Len=0 TSval=3167790385 TSecr=3167790385
9 0.026025958 127.0.0.1 → 127.0.0.1 TCP 66 80 → 35656 [FIN, ACK] Seq=10959 Ack=75 Win=65536 Len=0 TSval=3167790386 TSecr=3167790386
10 0.026083665 127.0.0.1 → 127.0.0.1 TCP 66 35656 → 80 [ACK] Seq=75 Ack=10960 Win=109440 Len=0 TSval=3167790386 TSecr=3167790386
```

Fig 2.0 Easier visual view.

The normal HTTP baseline showed a complete TCP three-way handshake. Frame 1 contained the initial SYN from source port 35656 to TCP port 80, Frame 2 contained Apache's SYN-ACK response, and Frame 3 completed the handshake with the client's ACK. An HTTP GET request followed in Frame 4 and the server returned HTTP/1.1 200 OK in Frame 6, confirming a successfully established application session.

Explanation

This view makes the complete connection easier to follow by showing the handshake together with the later HTTP traffic. The HTTP GET and 200 OK responses confirm that the TCP session was successfully established.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware-amd64
kali@kali: ~/SBT-DF203-Lab3
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab3]
$ cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

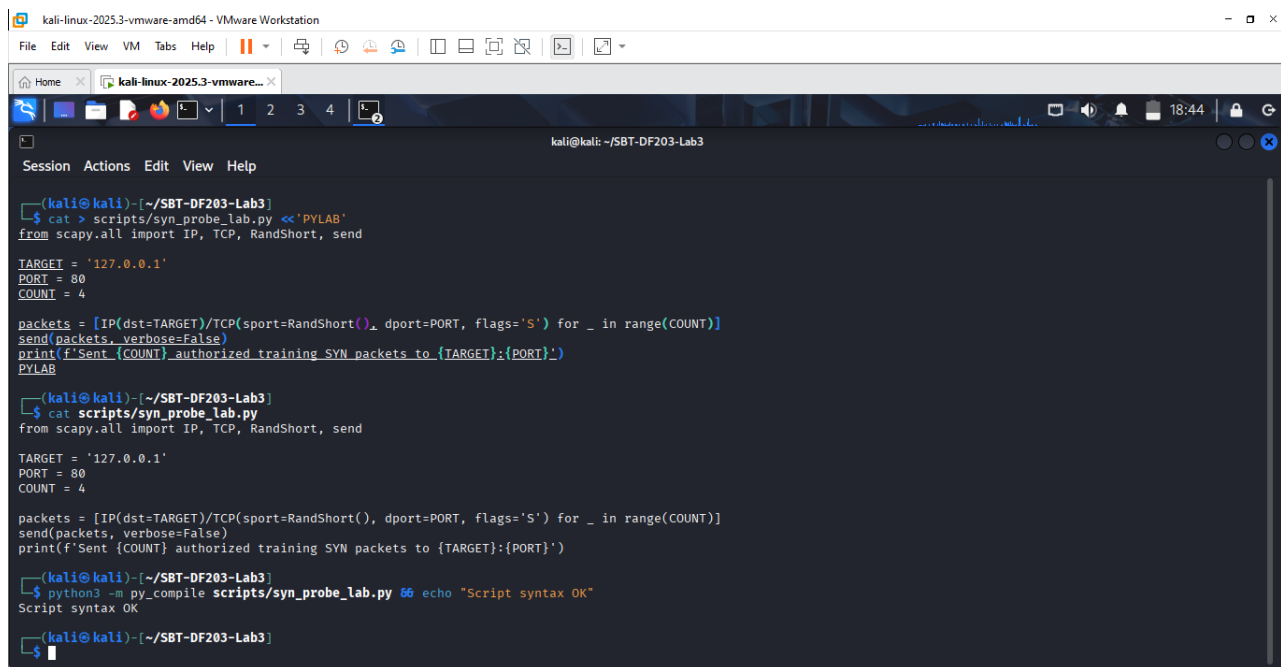
TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB
(kali@kali)-[~/SBT-DF203-Lab3]
$
```

Fig 2.1 Verifying the saved script.

Explanation

This screenshot shows the bounded Scapy script prepared for the practical. The target is 127.0.0.1, the destination port is 80 and the packet count is restricted to four, keeping the test within the authorised limits.



```
kali@kali: ~/SBT-DF203-Lab3
$ cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

(kali@kali)~/SBT-DF203-Lab3
$ cat scripts/syn_probe_lab.py
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')

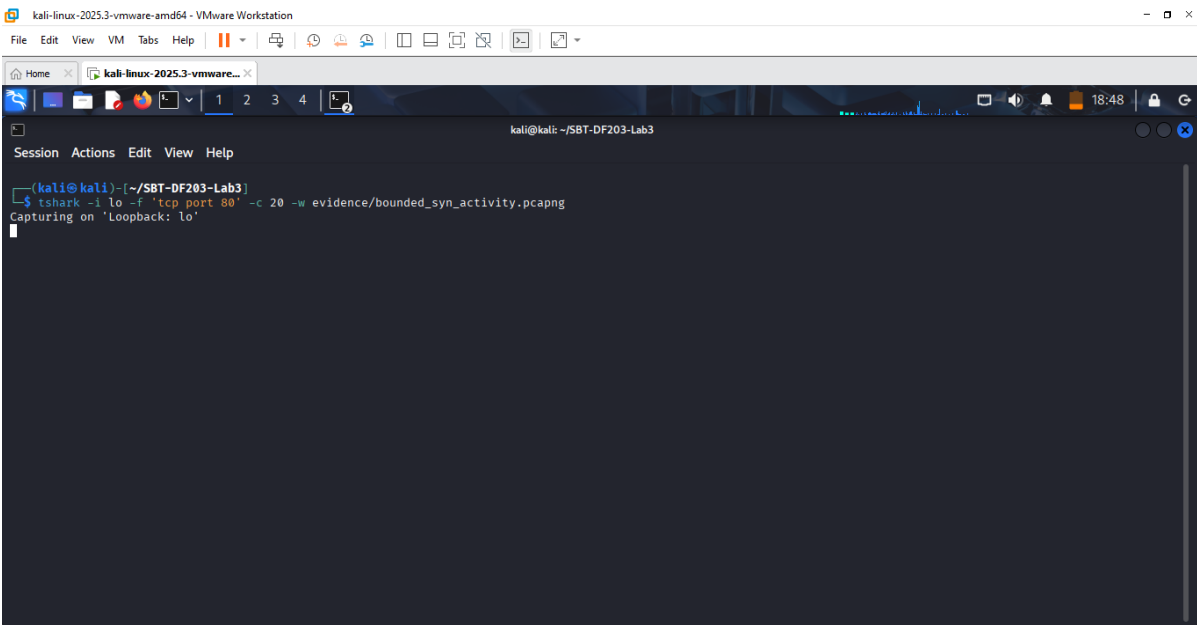
(kali@kali)~/SBT-DF203-Lab3
$ python3 -m py_compile scripts/syn_probe_lab.py 66 echo "Script syntax OK"
Script syntax OK

(kali@kali)~/SBT-DF203-Lab3
$
```

Fig 2.2 Verifying the saved script.

Explanation

This screenshot confirms that the saved Scapy script matches the intended four-packet configuration and passed the syntax check. Checking it before execution reduced the risk of accidentally sending more traffic than authorised.



```
kali@kali: ~/SBT-DF203-Lab3
$ tshark -i lo -f 'tcp.port 80' -c 20 -w evidence/bounded_syn_activity.pcapng
Capturing on 'Loopback: lo'
```

Fig 2.3 Running and actively capturing on the loopback interface lo.

Explanation

This screenshot shows TShark actively monitoring the loopback interface for TCP port 80 traffic. The capture was started before the Scapy script so that the complete bounded sequence and server responses could be recorded.

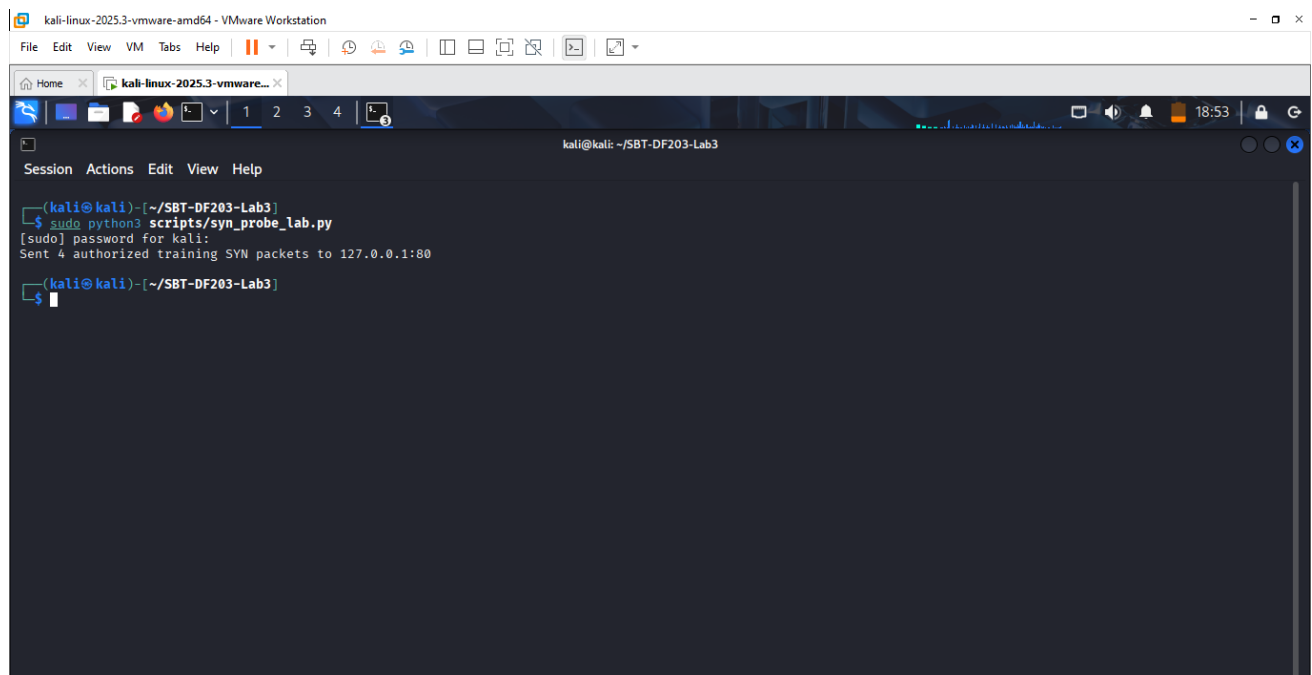


Fig 2.4 Sending the four authorized SYN packets.

Explanation

This screenshot records the execution of the Scapy script. The displayed message confirms that exactly four authorised SYN packets were sent to 127.0.0.1:80.

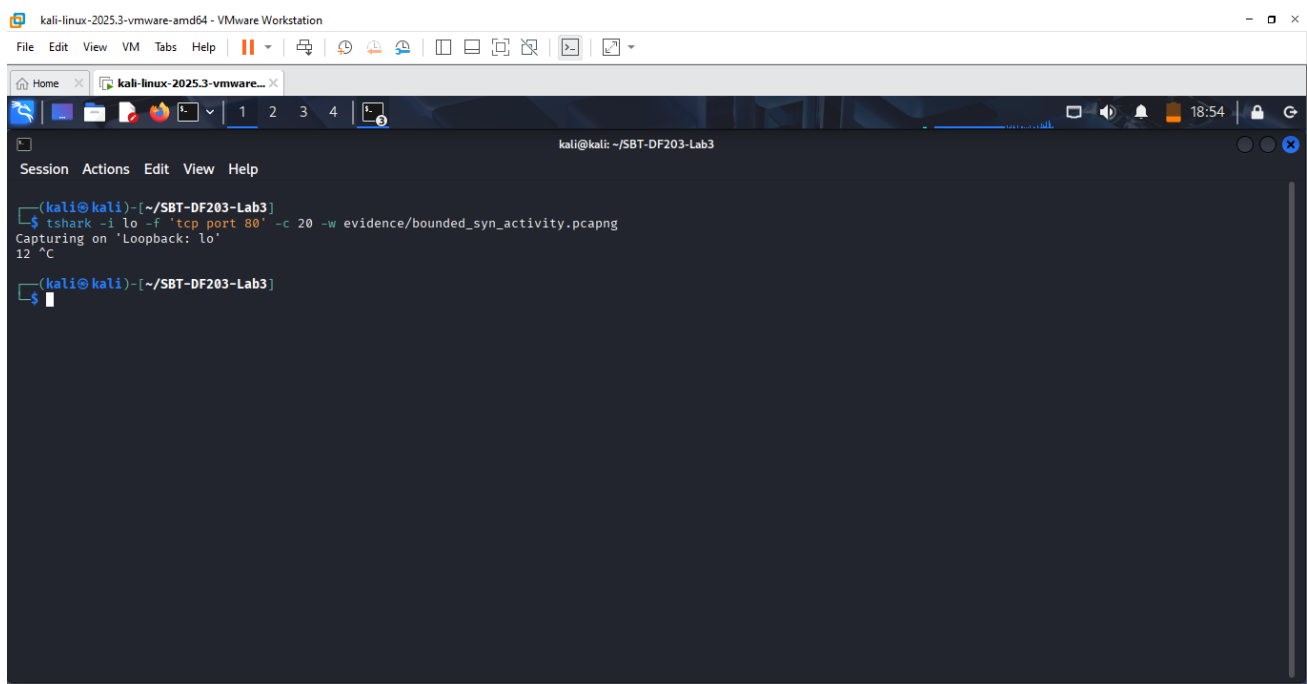
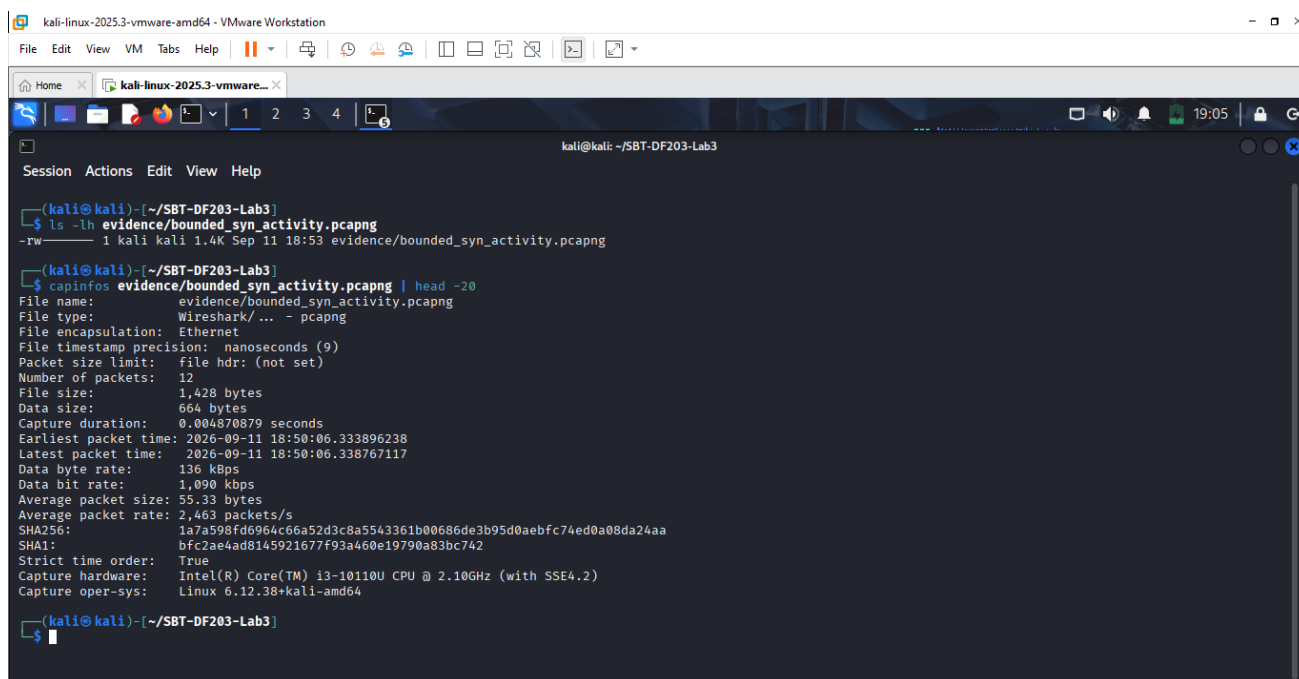


Fig 2.5 Packets captured successfully.

Explanation

This screenshot shows that the packet capture received the traffic generated during the simulation. The capture was then stopped once the four SYN attempts and their associated responses had been recorded.



```
kali@kali: ~/SBT-DF203-Lab3
$ ls -lh evidence/bounded_syn_activity.pcapng
-rw-r--r-- 1 kali kali 1.4K Sep 11 18:53 evidence/bounded_syn_activity.pcapng

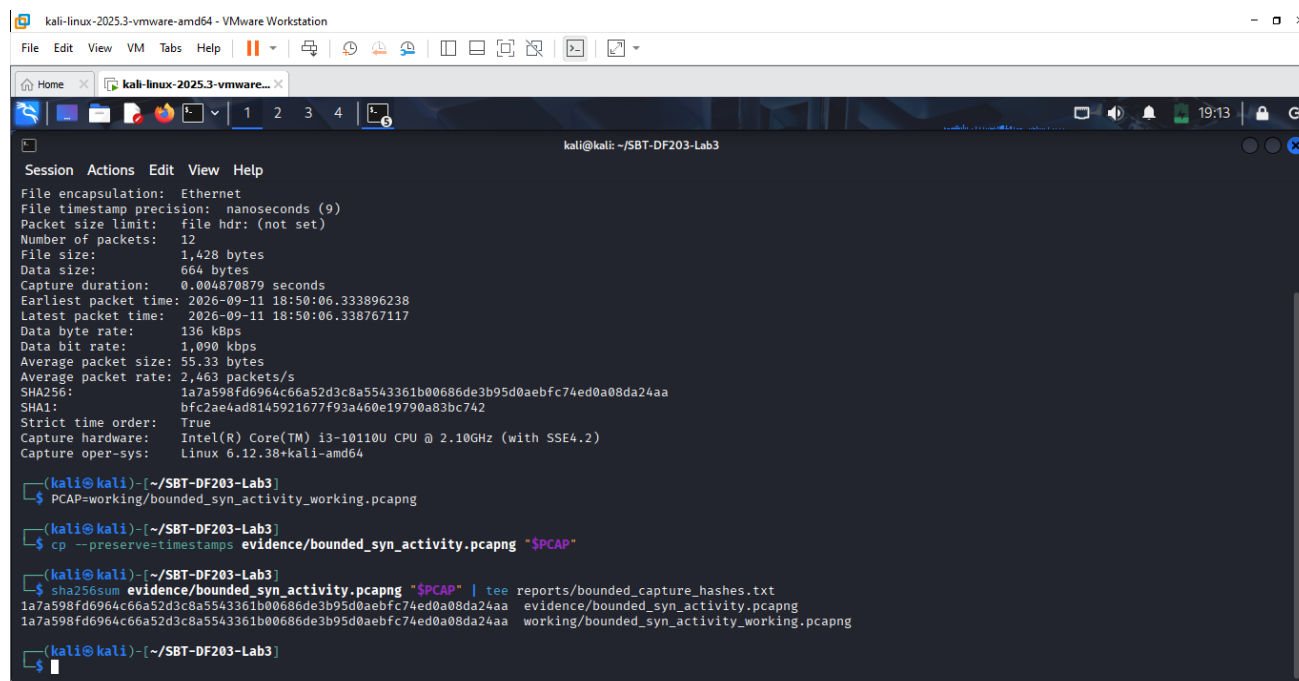
(kali@kali)~/SBT-DF203-Lab3
$ capinfos evidence/bounded_syn_activity.pcapng | head -20
File name:          evidence/bounded_syn_activity.pcapng
File type:          Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: nanoseconds (9)
Packet size limit:  file hdr: (not set)
Number of packets:  12
File size:          1,428 bytes
Data size:          664 bytes
Capture duration:   0.004870879 seconds
Earliest packet time: 2026-09-11 18:50:06.333896238
Latest packet time:  2026-09-11 18:50:06.338767117
Data byte rate:     136 kbps
Data bit rate:      1,090 kbps
Average packet size: 55.33 bytes
Average packet rate: 2,463 packets/s
SHA256:             1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa
SHA1:               bfc2ae4ad8145921677f93a460e19790a83bc742
Strict time order:  True
Capture hardware:   Intel(R) Core(TM) i3-10110U CPU @ 2.10GHz (with SSE4.2)
Capture oper-sys:   Linux 6.12.38+kali-amd64

(kali@kali)~/SBT-DF203-Lab3
$
```

Fig 2.6 Listing the bounded_syn_activity and details.

Explanation

This screenshot provides the basic properties of bounded_syn_activity.pcapng. It confirms that the capture contained 12 packets and records its size, timing and other file information.



```
kali@kali: ~/SBT-DF203-Lab3
$ PCAP=working/bounded_syn_activity_working.pcapng

(kali@kali)~/SBT-DF203-Lab3
$ cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"

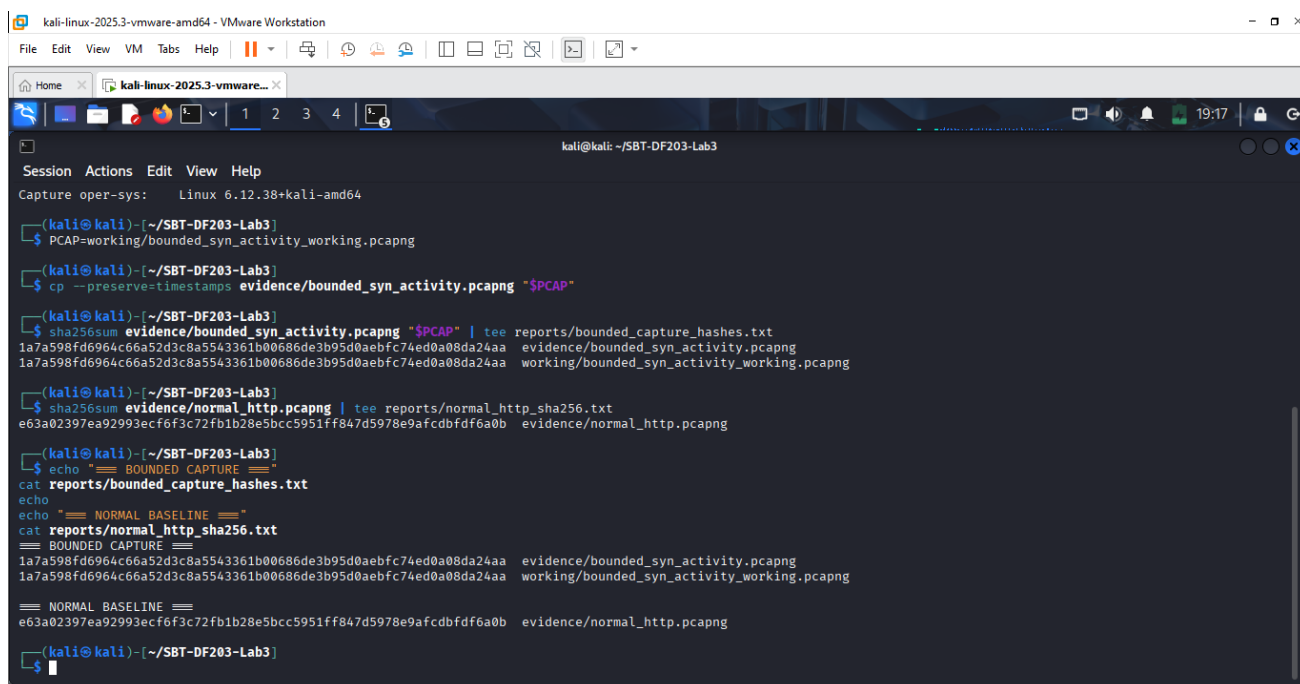
(kali@kali)~/SBT-DF203-Lab3
$ sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa evidence/bounded_syn_activity.pcapng
1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa working/bounded_syn_activity_working.pcapng

(kali@kali)~/SBT-DF203-Lab3
$
```

Fig 2.7 Preserving and hashing the both copies.

Explanation

This screenshot shows the original bounded capture being copied into the working directory for analysis. The identical SHA-256 values confirm that the working copy was byte-for-byte identical to the preserved original.

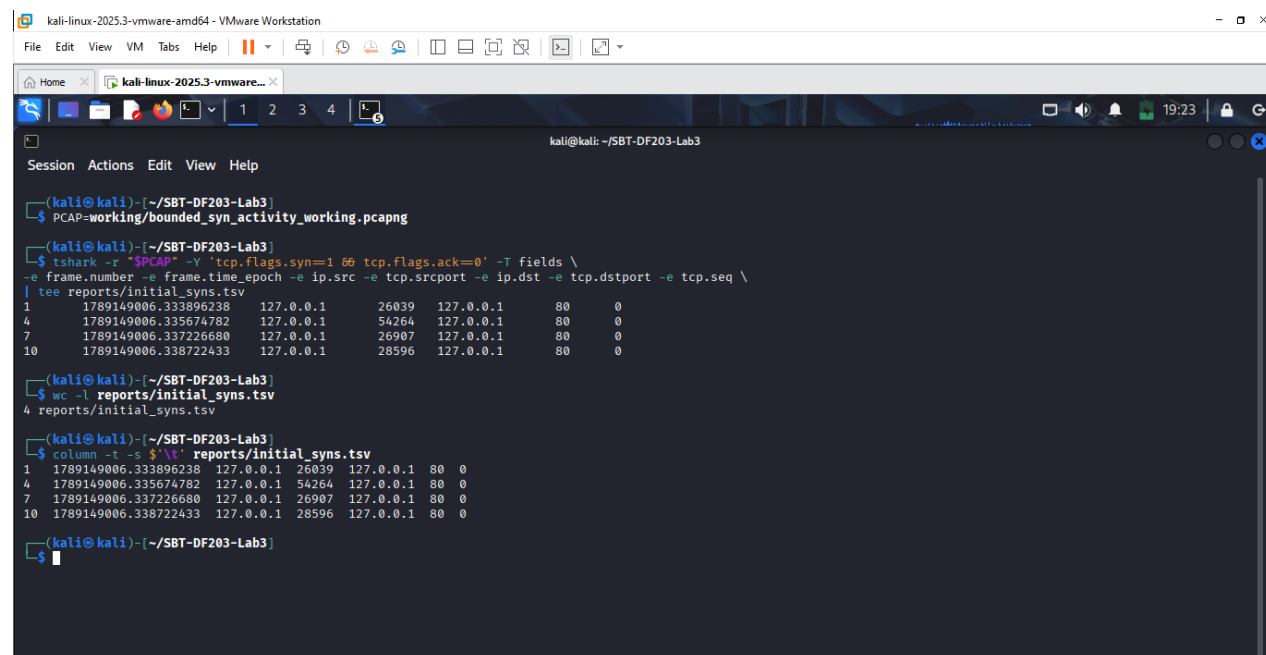


```
kali@kali: ~/SBT-DF203-Lab3
$ cp --preserve-timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
$ sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa evidence/bounded_syn_activity.pcapng
1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa working/bounded_syn_activity_working.pcapng
$ sha256sum evidence/normal_http.pcapng | tee reports/normal_http_sha256.txt
e63a02397ea92993ecf6f3c72fb1b28e5bcc5951ff847d5978e9afcdbfd6a0b evidence/normal_http.pcapng
$ echo "=== BOUNDED CAPTURE ==="
$ cat reports/bounded_capture_hashes.txt
$ echo "=== NORMAL BASELINE ==="
$ cat reports/normal_http_sha256.txt
=== BOUNDED CAPTURE ===
1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa evidence/bounded_syn_activity.pcapng
1a7a598fd6964c66a52d3c8a5543361b00686de3b95d0aebfc74ed0a08da24aa working/bounded_syn_activity_working.pcapng
=== NORMAL BASELINE ===
e63a02397ea92993ecf6f3c72fb1b28e5bcc5951ff847d5978e9afcdbfd6a0b evidence/normal_http.pcapng
```

Fig 2.8 Hashing the normal baseline.

Explanation

This screenshot records the SHA-256 value of the normal HTTP baseline and displays the previously calculated bounded-capture hashes. These values provide integrity references for the evidence used in the report.



```
kali@kali: ~/SBT-DF203-Lab3
$ PCAP=working/bounded_syn_activity_working.pcapng
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
| tee reports/initial_syns.tsv
1 1789149006.333896238 127.0.0.1 26039 127.0.0.1 80 0
4 1789149006.335674782 127.0.0.1 54264 127.0.0.1 80 0
7 1789149006.337226680 127.0.0.1 26907 127.0.0.1 80 0
10 1789149006.338722433 127.0.0.1 28596 127.0.0.1 80 0
$ wc -l reports/initial_syns.tsv
4 reports/initial_syns.tsv
$ column -t -s $'\t' reports/initial_syns.tsv
1 1789149006.333896238 127.0.0.1 26039 127.0.0.1 80 0
4 1789149006.335674782 127.0.0.1 54264 127.0.0.1 80 0
7 1789149006.337226680 127.0.0.1 26907 127.0.0.1 80 0
10 1789149006.338722433 127.0.0.1 28596 127.0.0.1 80 0
```

Fig 2.9 Initialing SYN Packets.

Frame	Epoch time	Source	Source port	Destination	Dst port	Seq
1	1789149006.333896238	127.0.0.1	26039	127.0.0.1	80	0
4	1789149006.335674782	127.0.0.1	54264	127.0.0.1	80	0
7	1789149006.337226680	127.0.0.1	26907	127.0.0.1	80	0
10	1789149006.338722433	127.0.0.1	28596	127.0.0.1	80	0

Explanation

This screenshot applies the TShark filter for SYN packets where the ACK flag is not set. Four initial SYNs were identified, using source ports 26039, 54264, 26907 and 28596, all directed to TCP port 80.

```

kali@kali: ~/SBT-DF203-Lab3
$ PCAP=working/bounded_syn_activity_working.pcapng
$ tshark -r "$PCAP" -Y 'tcp.flags.syn=1 && tcp.flags.ack=1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
| tee reports/syn_ack_responses.tsv
2 1789149006.333985442 127.0.0.1 80 127.0.0.1 26039 1
5 1789149006.335715556 127.0.0.1 80 127.0.0.1 54264 1
8 1789149006.337258909 127.0.0.1 80 127.0.0.1 26907 1
11 1789149006.338755572 127.0.0.1 80 127.0.0.1 28596 1
$ wc -l reports/syn_ack_responses.tsv
4 reports/syn_ack_responses.tsv
$ column -t -s $'\t' reports/syn_ack_responses.tsv
2 1789149006.333985442 127.0.0.1 80 127.0.0.1 26039 1
5 1789149006.335715556 127.0.0.1 80 127.0.0.1 54264 1
8 1789149006.337258909 127.0.0.1 80 127.0.0.1 26907 1
11 1789149006.338755572 127.0.0.1 80 127.0.0.1 28596 1

```

Fig 3.0 SYN ACK Responses.

Frame	Source	Source Port	Destination	Destination Port	ACK
2	127.0.0.1	80	127.0.0.1	26039	1
5	127.0.0.1	80	127.0.0.1	54264	1
8	127.0.0.1	80	127.0.0.1	26907	1
11	127.0.0.1	80	127.0.0.1	28596	1

Explanation

This screenshot shows four SYN-ACK responses generated by Apache. Each response corresponds to one of the four initial SYN packets, confirming that the web server received and answered every connection attempt.

```

kali@kali:~/SBT-DF203-Lab3
$ PCAP=working/bounded_syn_activity_working.pcapng

(kali@kali)~/SBT-DF203-Lab3
$ tshark -r "$PCAP" -Y 'tcp.flags.reset=1 || (tcp.flags.ack=1 && tcp.len=0)' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/ack_reset_candidates.tsv
 1 1789149006.333985442 127.0.0.1 80 127.0.0.1 26039 0x0012
 2 1789149006.334011405 127.0.0.1 26039 127.0.0.1 80 0x0004
 3 1789149006.335715556 127.0.0.1 80 127.0.0.1 54264 0x0012
 4 1789149006.335728781 127.0.0.1 54264 127.0.0.1 80 0x0004
 5 1789149006.337258909 127.0.0.1 80 127.0.0.1 26907 0x0012
 6 1789149006.337270673 127.0.0.1 26907 127.0.0.1 80 0x0004
 7 1789149006.338755572 127.0.0.1 80 127.0.0.1 28596 0x0012
 8 1789149006.338767117 127.0.0.1 28596 127.0.0.1 80 0x0004

(kali@kali)~/SBT-DF203-Lab3
$ column -t -s $'\t' reports/ack_reset_candidates.tsv
 1 1789149006.333985442 127.0.0.1 80 127.0.0.1 26039 0x0012
 2 1789149006.334011405 127.0.0.1 26039 127.0.0.1 80 0x0004
 3 1789149006.335715556 127.0.0.1 80 127.0.0.1 54264 0x0012
 4 1789149006.335728781 127.0.0.1 54264 127.0.0.1 80 0x0004
 5 1789149006.337258909 127.0.0.1 80 127.0.0.1 26907 0x0012
 6 1789149006.337270673 127.0.0.1 26907 127.0.0.1 80 0x0004
 7 1789149006.338755572 127.0.0.1 80 127.0.0.1 28596 0x0012
 8 1789149006.338767117 127.0.0.1 28596 127.0.0.1 80 0x0004

```

Fig 3.1 Examining ACK and RST behaviour.

Explanation

This screenshot examines packets that could represent ACK or reset behaviour following the initial SYN exchanges. It provided the first indication that the simulated sessions were not completing in the same way as the normal baseline.

```

kali@kali:~/SBT-DF203-Lab3
$ tshark -r "$PCAP" -Y 'tcp.flags.reset=1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport \
-e ip.dst -e tcp.dstport -e tcp.flags -e _ws.col.info
 1 0.000115167 127.0.0.1 26039 127.0.0.1 80 0x0004 26039 → 80 [RST] Seq=1 Win=0 Len=0
 2 0.001832543 127.0.0.1 54264 127.0.0.1 80 0x0004 54264 → 80 [RST] Seq=1 Win=0 Len=0
 3 0.003374435 127.0.0.1 26907 127.0.0.1 80 0x0004 26907 → 80 [RST] Seq=1 Win=0 Len=0
 4 0.004870879 127.0.0.1 28596 127.0.0.1 80 0x0004 28596 → 80 [RST] Seq=1 Win=0 Len=0

(kali@kali)~/SBT-DF203-Lab3
$ tshark -r "$PCAP" -Y 'tcp.flags.reset=1' -T fields -e frame.number | wc -l
4

(kali@kali)~/SBT-DF203-Lab3
$ tshark -r "$PCAP" -Y 'tcp.port==80'
 1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 54 26039 → 80 [SYN] Seq=0 Win=8192 Len=0
 2 0.000089204 127.0.0.1 → 127.0.0.1 TCP 58 80 → 26039 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
 3 0.000115167 127.0.0.1 → 127.0.0.1 TCP 54 26039 → 80 [RST] Seq=1 Win=0 Len=0
 4 0.001778844 127.0.0.1 → 127.0.0.1 TCP 54 54264 → 80 [SYN] Seq=0 Win=8192 Len=0
 5 0.001819318 127.0.0.1 → 127.0.0.1 TCP 58 80 → 54264 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
 6 0.001832543 127.0.0.1 → 127.0.0.1 TCP 54 54264 → 80 [RST] Seq=1 Win=0 Len=0
 7 0.003330442 127.0.0.1 → 127.0.0.1 TCP 54 26907 → 80 [SYN] Seq=0 Win=8192 Len=0
 8 0.003362671 127.0.0.1 → 127.0.0.1 TCP 58 80 → 26907 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
 9 0.003374435 127.0.0.1 → 127.0.0.1 TCP 54 26907 → 80 [RST] Seq=1 Win=0 Len=0
10 0.004826195 127.0.0.1 → 127.0.0.1 TCP 54 28596 → 80 [SYN] Seq=0 Win=8192 Len=0
11 0.004859334 127.0.0.1 → 127.0.0.1 TCP 58 80 → 28596 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
12 0.004870879 127.0.0.1 → 127.0.0.1 TCP 54 28596 → 80 [RST] Seq=1 Win=0 Len=0

```

Fig 3.2 ACK RST Behaviour page 2.

Explanation

This screenshot clearly shows four reset packets in Frames 3, 6, 9 and 12. The complete sequence demonstrates that every attempt followed SYN → SYN-ACK → RST rather than ending with the normal final ACK.

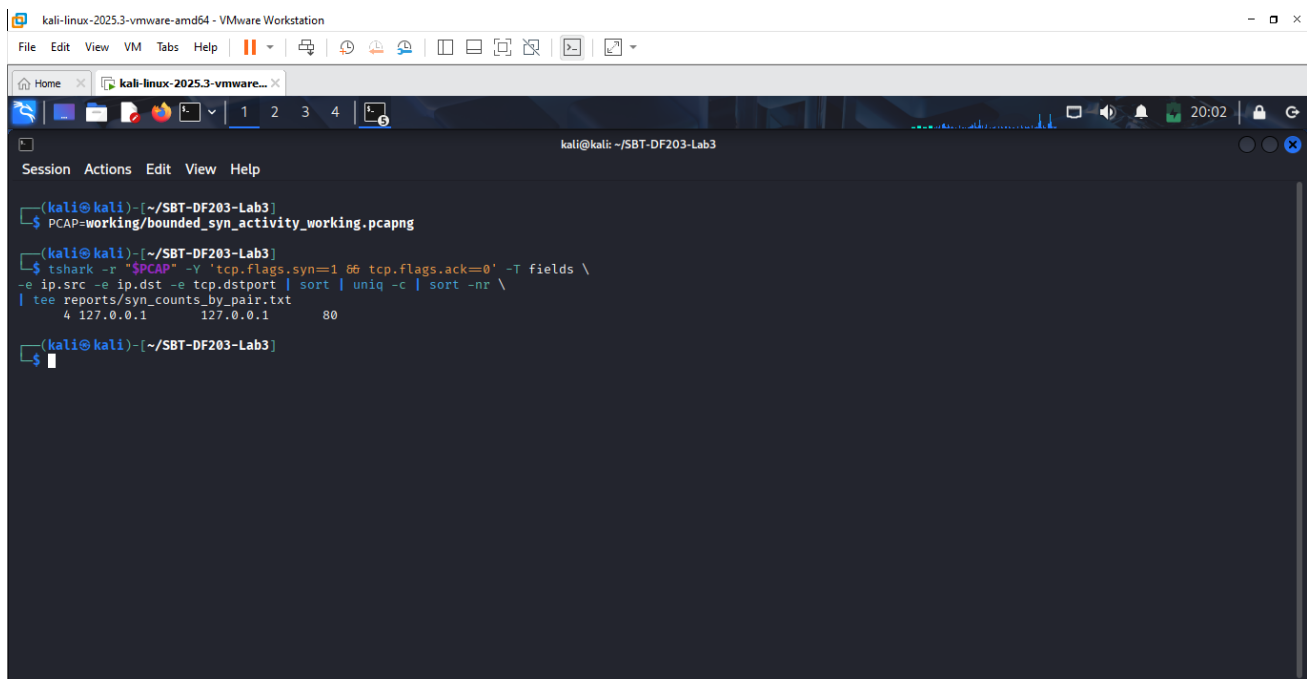


Fig 3.3 Count SYNs by source/destination.

Explanation

This screenshot counts the initial SYN packets according to source, destination and destination port. The result confirms four SYN packets from 127.0.0.1 to the local Apache service on TCP port 80.

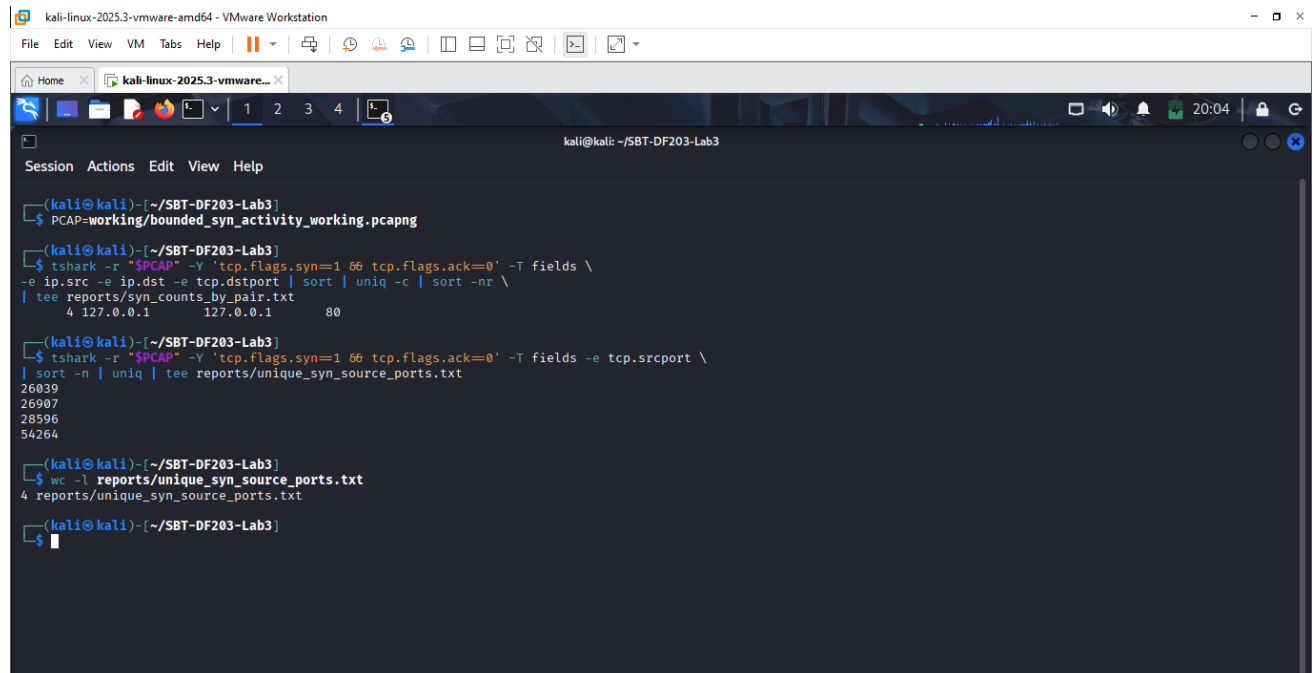


Fig 3.4 Extract unique SYN source ports.

Explanation

This screenshot extracts the client-side source ports used by the four SYN packets. Four different source ports were identified, confirming that each simulated connection attempt used a separate source port.

```
kali@kali: ~/SBT-DF203-Lab3
$ wc -l reports/unique_syn_source_ports.txt
4 reports/unique_syn_source_ports.txt

(kali@kali)~/SBT-DF203-Lab3
$ echo "=== SYN COUNTS BY SOURCE / DESTINATION ==="
cat reports/syn_counts_by_pair.txt
echo
echo "=== UNIQUE SYN SOURCE PORTS ==="
cat reports/unique_syn_source_ports.txt
echo
echo "=== UNIQUE PORT COUNT ==="
wc -l reports/unique_syn_source_ports.txt
=== SYN COUNTS BY SOURCE / DESTINATION ===
4 127.0.0.1 127.0.0.1 80

=== UNIQUE SYN SOURCE PORTS ===
26039
26907
28596
54264

=== UNIQUE PORT COUNT ===
4 reports/unique_syn_source_ports.txt

(kali@kali)~/SBT-DF203-Lab3
```

Fig 3.5 Displaying both findings together.

Explanation

This screenshot brings the SYN count and unique source-port results together in one view. It confirms four initial SYN packets and four unique client-side source ports targeting the same Apache service.

```
kali@kali: ~/SBT-DF203-Lab3
$ PCAP=working/bounded_syn_activity_working.pcapng
$ tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt

Warns (4)
-----
Frequency  Group      Protocol Summary
4          Sequence  TCP      Connection reset (RST)

Notes (4)
-----
Frequency  Group      Protocol Summary
4          Protocol  TCP      The SYN packet does not contain a SACK PERM option

Chats (8)
-----
Frequency  Group      Protocol Summary
4          Sequence  TCP      Connection establish request (SYN): server port 80
4          Sequence  TCP      Connection establish acknowledge (SYN+ACK): server port 80

(kali@kali)~/SBT-DF203-Lab3
```

Fig 3.6 TShark Expert Information.

Explanation

This screenshot shows TShark's Expert Information for the bounded capture. It identified four connection resets along with four SYN requests and four SYN-ACK responses, which agrees with the packet-level analysis.

```
kali@kali: ~/SBT-DF203-Lab3
$ tshark -r "$PCAP" \
-Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
-T fields \
-e frame.number -e frame.time -e _ws.col.Info \
| tee reports/tcp_analysis_events.tsv

(kali@kali)~/SBT-DF203-Lab3
$ wc -l reports/tcp_analysis_events.tsv
0 reports/tcp_analysis_events.tsv

(kali@kali)~/SBT-DF203-Lab3
$ cat reports/expert_info.txt

Warns (4)
=====
Frequency  Group      Protocol Summary
4          Sequence    TCP      Connection reset (RST)

Notes (4)
=====
Frequency  Group      Protocol Summary
4          Protocol   TCP      The SYN packet does not contain a SACK PERM option

Chats (8)
=====
Frequency  Group      Protocol Summary
4          Sequence   TCP      Connection establish request (SYN): server port 80
4          Sequence   TCP      Connection establish acknowledge (SYN+ACK): server port 80

(kali@kali)~/SBT-DF203-Lab3
$
```

Fig 3.7 TShark Expert TCP Analysis.

Explanation

This screenshot shows the additional check for retransmissions, lost segments and duplicate ACKs. No packets matched those conditions, while the Expert Information continued to show the four connection resets.

```
kali@kali: ~/SBT-DF203-Lab3
$ echo "=== NORMAL BASELINE ==="

echo -n "Initial SYNs: "
tshark -r evidence/normal_http.pcapng \
-Y 'tcp.flags.syn==1 && tcp.flags.ack==0' \
-T fields -e frame.number | wc -l

echo -n "SYN-ACKs: "
tshark -r evidence/normal_http.pcapng \
-Y 'tcp.flags.syn==1 && tcp.flags.ack==1' \
-T fields -e frame.number | wc -l

echo -n "RST packets: "
tshark -r evidence/normal_http.pcapng \
-Y 'tcp.flags.reset==1' \
-T fields -e frame.number | wc -l

echo -n "HTTP requests: "
tshark -r evidence/normal_http.pcapng \
-Y 'http.request' \
-T fields -e frame.number | wc -l

=== NORMAL BASELINE ===
Initial SYNs: 1
SYN-ACKs: 1
RST packets: 0
HTTP requests: 1

(kali@kali)~/SBT-DF203-Lab3
$
```

Fig 3.8 Verifying the normal baseline.

Explanation

This screenshot quantifies the normal HTTP session. It confirms one initial SYN, one SYN-ACK, no RST packets and one HTTP request, which is consistent with a successfully established connection.

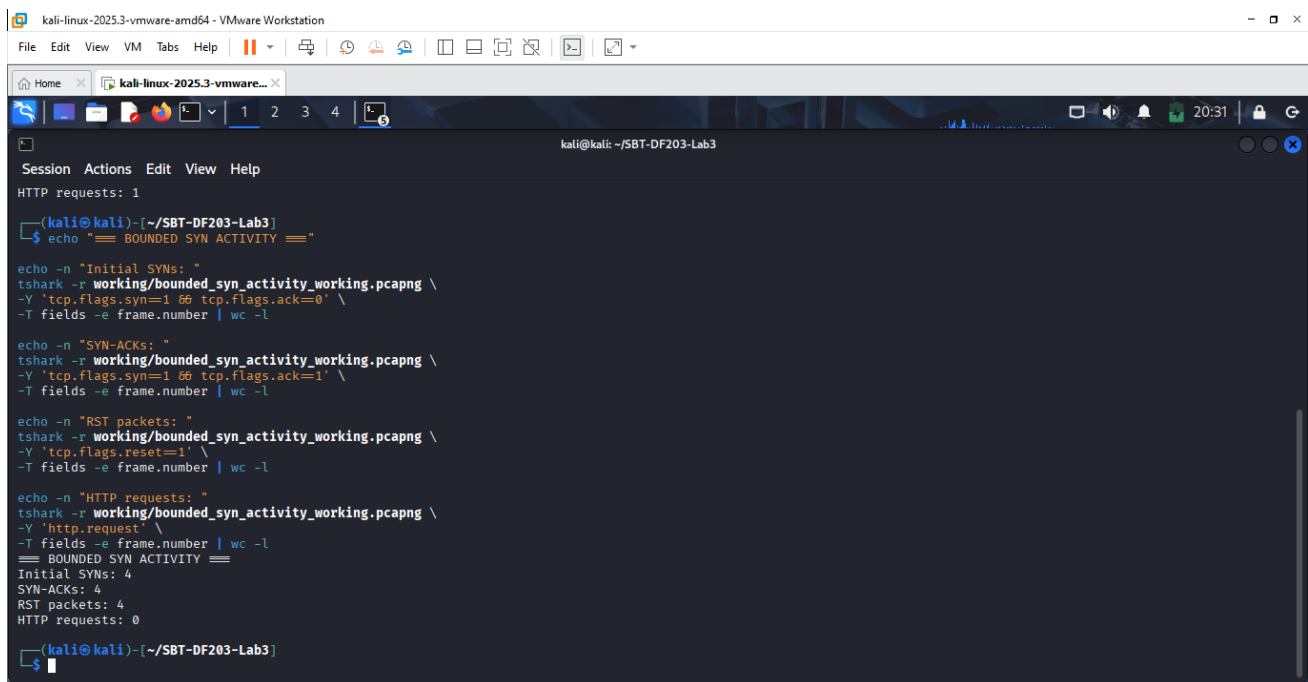


Fig 3.9 Verifying the bounded capture.

Explanation

This screenshot applies the same measurements to the bounded activity. It confirms four SYNs, four SYN-ACKs, four RST packets and no HTTP request, highlighting the difference from the legitimate baseline.

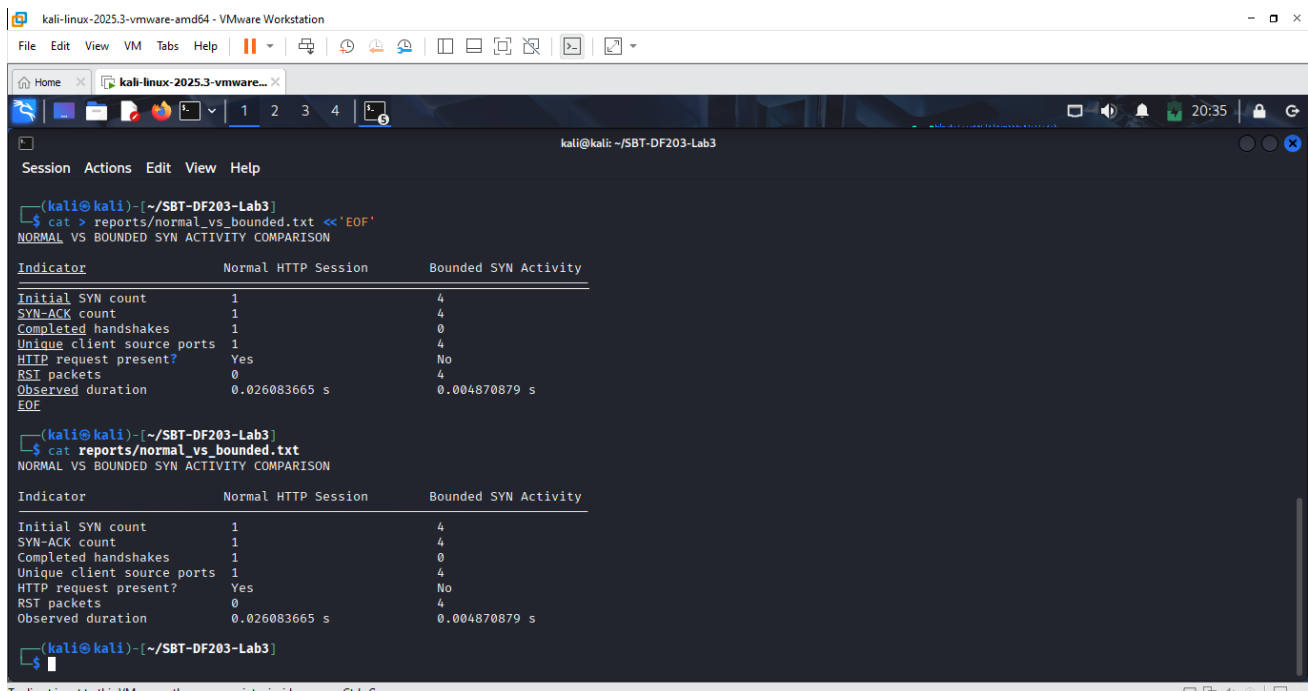


Fig 4.0 Comparison table.

Indicator	Normal HTTP	Bounded SYN Activity
Initial SYNs	1	4
SYN-ACKs	1	4
Completed handshakes	1	0
Unique client source ports	1	4
HTTP request	Yes	No
RST packets	0	4
Observed duration	0.026083665 s	0.004870879 s

Explanation

This screenshot directly compares the normal HTTP connection with the bounded SYN activity. The normal session completed one handshake and generated HTTP traffic, while the bounded capture completed none and produced four resets.

```

(kali@kali)~/SBT-DF203-Lab3
$ PCAP=working/bounded_syn_activity_working.pcapng

(kali@kali)~/SBT-DF203-Lab3
$ tshark -r "$PCAP" -Y 'tcp.port==80' -T fields -E header=y \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags \
-e _ws.col.info \
| tee reports/forensic_timeline.tsv
frame.number  frame.time_relative  ip.src  tcp.srcport  ip.dst  tcp.dstport  tcp.flags  _ws.col.info
1  0.000000000  127.0.0.1  26039  127.0.0.1  80  0x0002  26039 → 80 [SYN] Seq=0 Win=8192 Len=0
2  0.000089204  127.0.0.1  80  127.0.0.1  26039  0x0012  80 → 26039 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
3  0.000115167  127.0.0.1  26039  127.0.0.1  80  0x0004  26039 → 80 [RST] Seq=1 Win=0 Len=0
4  0.001778544  127.0.0.1  54264  127.0.0.1  80  0x0002  54264 → 80 [SYN] Seq=0 Win=8192 Len=0
5  0.001819318  127.0.0.1  80  127.0.0.1  54264  0x0012  80 → 54264 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
6  0.001832543  127.0.0.1  54264  127.0.0.1  80  0x0004  54264 → 80 [RST] Seq=1 Win=0 Len=0
7  0.003330442  127.0.0.1  26907  127.0.0.1  80  0x0002  26907 → 80 [SYN] Seq=0 Win=8192 Len=0
8  0.003362671  127.0.0.1  80  127.0.0.1  26907  0x0012  80 → 26907 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
9  0.003374435  127.0.0.1  26907  127.0.0.1  80  0x0004  26907 → 80 [RST] Seq=1 Win=0 Len=0
10  0.004826195  127.0.0.1  28596  127.0.0.1  80  0x0002  28596 → 80 [SYN] Seq=0 Win=8192 Len=0
11  0.004859334  127.0.0.1  80  127.0.0.1  28596  0x0012  80 → 28596 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495
12  0.004870879  127.0.0.1  28596  127.0.0.1  80  0x0004  28596 → 80 [RST] Seq=1 Win=0 Len=0

```

Fig 4.1 Building the Forensic Timeline.

Explanation

This screenshot extracts the frame number, relative time, IP addresses, source and destination ports, TCP flags and packet information. These fields provide the evidence needed to reconstruct the order of events.

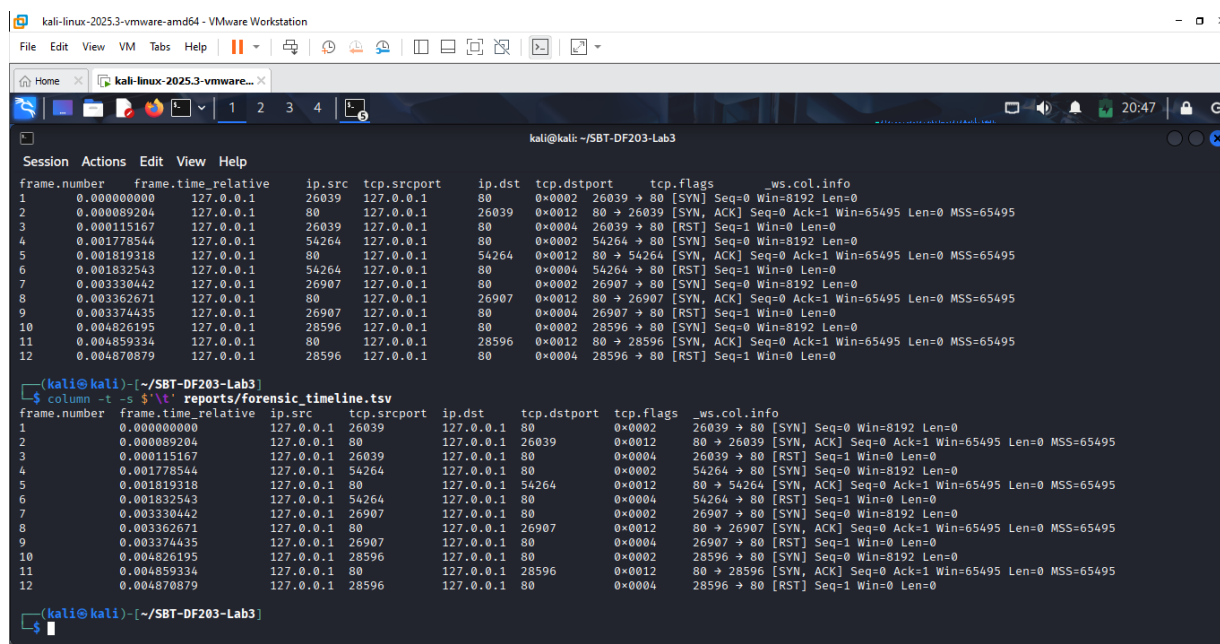


Fig 4.2 Building the Forensic Timeline page 1.

Explanation

This screenshot presents the 12 captured packets in chronological order. The repeating groups of SYN, SYN-ACK and RST make the incomplete connection pattern easy to identify.

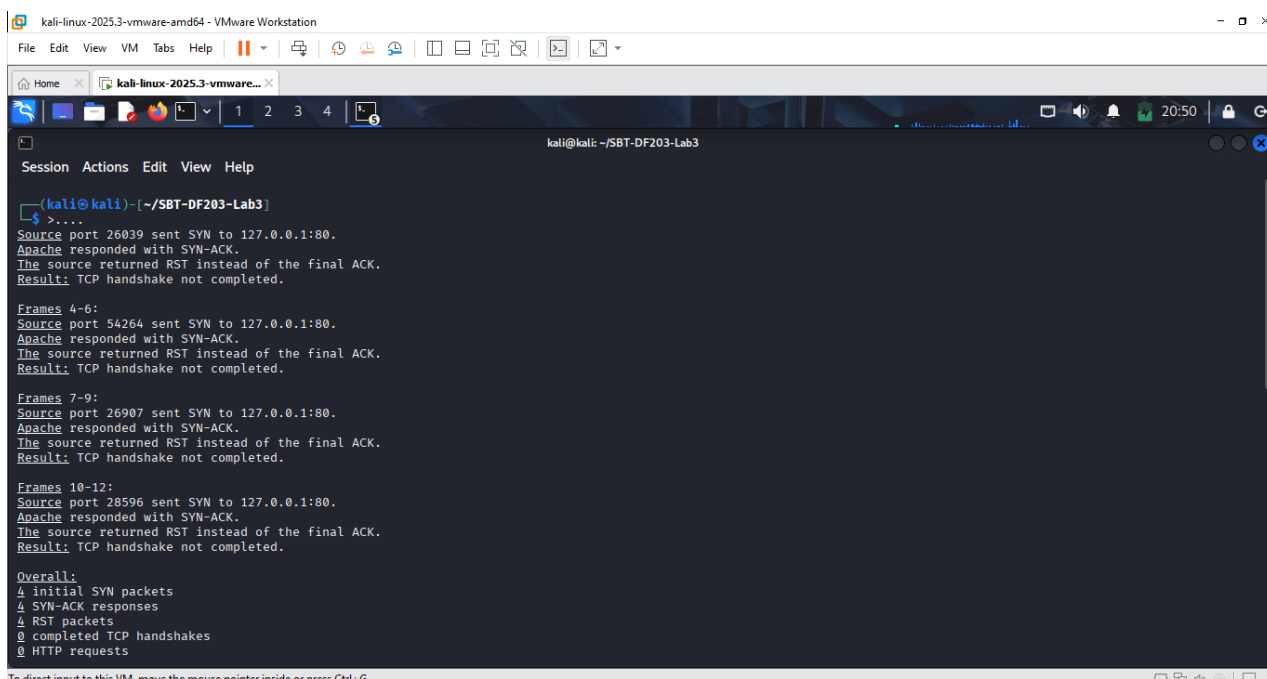


Fig 4.3 Forensic Timeline.

Explanation

This screenshot begins the written reconstruction of the four simulated connection attempts. Each group describes the initial SYN, Apache's SYN-ACK response and the RST that prevented the handshake from completing.

```
kali@kali: ~/SBT-DF203-Lab3
$ cat reports/forensic_timeline_summary.txt
FORENSIC TIMELINE - BOUNDED SYN ACTIVITY

Frames 1-3:
Source port 26039 sent SYN to 127.0.0.1:80.
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

Frames 4-6:
Source port 54264 sent SYN to 127.0.0.1:80.
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

Frames 7-9:
Source port 26907 sent SYN to 127.0.0.1:80.
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

Frames 10-12:
Source port 28596 sent SYN to 127.0.0.1:80.
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

Overall:
4 initial SYN packets
4 SYN-ACK responses
```

Fig 4.4 Forensic Timeline page 1.

Explanation

This screenshot continues the event-by-event summary for the four source ports. It shows that the same incomplete connection behaviour occurred consistently during each attempt.

```
kali@kali: ~/SBT-DF203-Lab3
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

Frames 4-6:
Source port 54264 sent SYN to 127.0.0.1:80.
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

Frames 7-9:
Source port 26907 sent SYN to 127.0.0.1:80.
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

Frames 10-12:
Source port 28596 sent SYN to 127.0.0.1:80.
Apache responded with SYN-ACK.
The source returned RST instead of the final ACK.
Result: TCP handshake not completed.

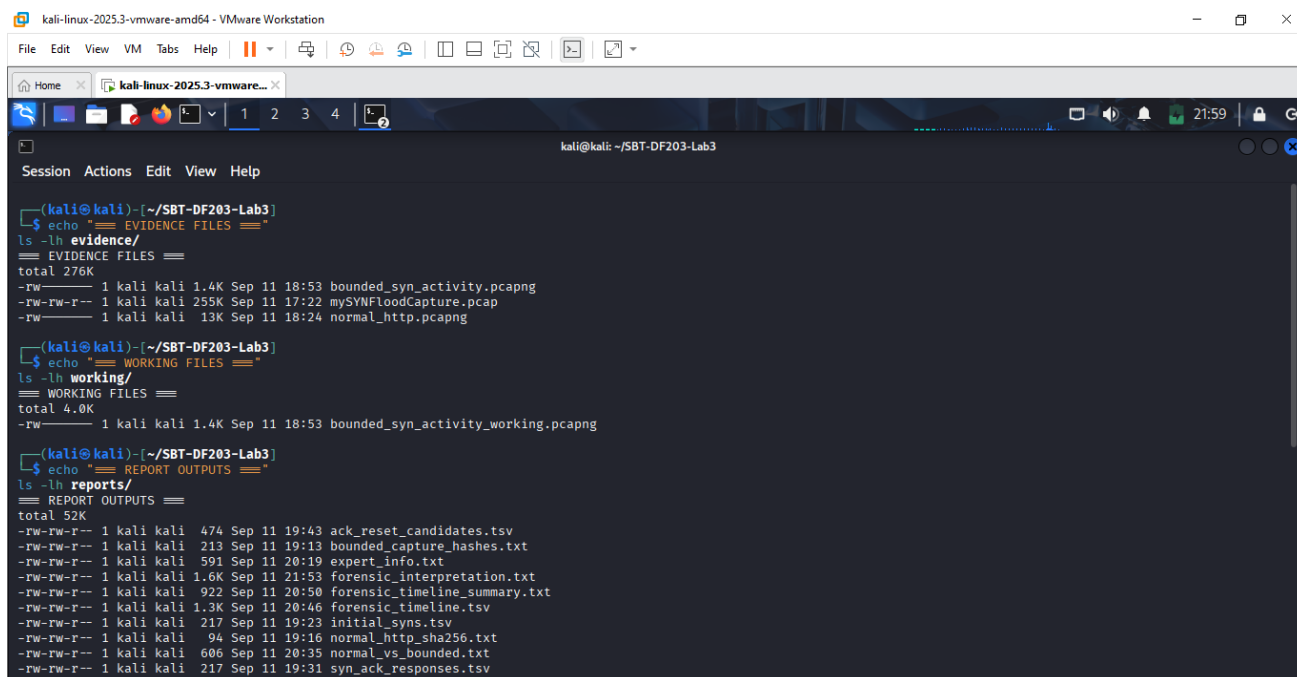
Overall:
4 initial SYN packets
4 SYN-ACK responses
4 RST packets
0 completed TCP handshakes
0 HTTP requests
Observed capture activity duration: approximately 0.004870879 seconds

(kali@kali) ~/SBT-DF203-Lab3
$
```

Fig 4.5 Forensic Timeline page 2.

Explanation

This screenshot summarises the overall timeline with four initial SYNs, four SYN-ACKs, four RSTs, zero completed handshakes and zero HTTP requests. It also records the approximately 0.004870879-second duration of the observed bounded activity.



```
kali@kali: ~/SBT-DF203-Lab3
$ echo "=== EVIDENCE FILES ==="
ls -lh evidence/
=== EVIDENCE FILES ===
total 276K
-rw-rw-r-- 1 kali kali 1.4K Sep 11 18:53 bounded_syn_activity.pcapng
-rw-rw-r-- 1 kali kali 255K Sep 11 17:22 mySYNFloodCapture.pcap
-rw-rw-r-- 1 kali kali 13K Sep 11 18:24 normal_http.pcapng

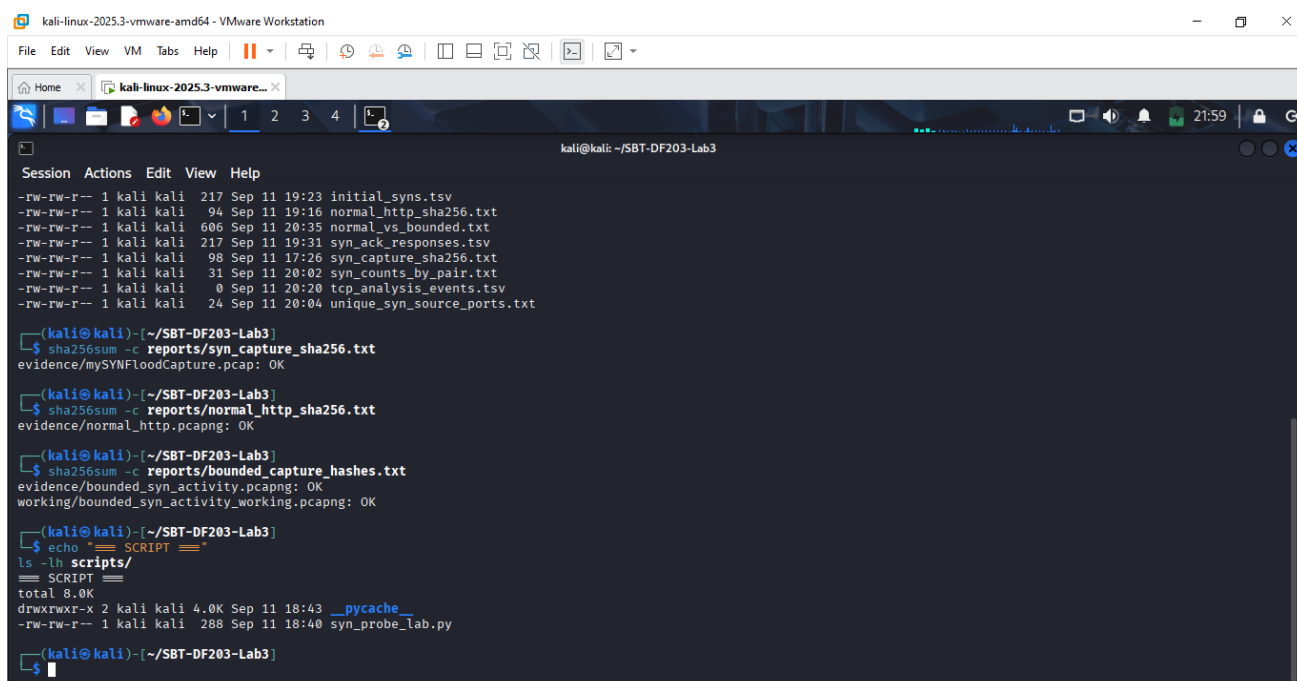
(kali@kali)~/SBT-DF203-Lab3
$ echo "=== WORKING FILES ==="
ls -lh working/
=== WORKING FILES ===
total 4.0K
-rw-rw-r-- 1 kali kali 1.4K Sep 11 18:53 bounded_syn_activity_working.pcapng

(kali@kali)~/SBT-DF203-Lab3
$ echo "=== REPORT OUTPUTS ==="
ls -lh reports/
=== REPORT OUTPUTS ===
total 52K
-rw-rw-r-- 1 kali kali 474 Sep 11 19:43 ack_reset_candidates.tsv
-rw-rw-r-- 1 kali kali 213 Sep 11 19:13 bounded_capture_hashes.txt
-rw-rw-r-- 1 kali kali 591 Sep 11 20:19 expert_info.txt
-rw-rw-r-- 1 kali kali 1.6K Sep 11 21:53 forensic_interpretation.txt
-rw-rw-r-- 1 kali kali 922 Sep 11 20:50 forensic_timeline_summary.txt
-rw-rw-r-- 1 kali kali 1.3K Sep 11 20:46 forensic_timeline.tsv
-rw-rw-r-- 1 kali kali 217 Sep 11 19:23 initial_syns.tsv
-rw-rw-r-- 1 kali kali 94 Sep 11 19:16 normal_http_sha256.txt
-rw-rw-r-- 1 kali kali 606 Sep 11 20:35 normal_vs_bounded.txt
-rw-rw-r-- 1 kali kali 217 Sep 11 19:31 syn_ack_responses.tsv
```

Fig 4.6 Check all analysis outputs.

Explanation

This screenshot shows the contents of the evidence, working and reports directories. It demonstrates that the main capture files and TShark analysis outputs were retained after the practical.



```
kali@kali: ~/SBT-DF203-Lab3
-rw-rw-r-- 1 kali kali 217 Sep 11 19:23 initial_syns.tsv
-rw-rw-r-- 1 kali kali 94 Sep 11 19:16 normal_http_sha256.txt
-rw-rw-r-- 1 kali kali 606 Sep 11 20:35 normal_vs_bounded.txt
-rw-rw-r-- 1 kali kali 217 Sep 11 19:31 syn_ack_responses.tsv
-rw-rw-r-- 1 kali kali 98 Sep 11 17:26 syn_capture_sha256.txt
-rw-rw-r-- 1 kali kali 31 Sep 11 20:02 syn_counts_by_pair.txt
-rw-rw-r-- 1 kali kali 0 Sep 11 20:20 tcp_analysis_events.tsv
-rw-rw-r-- 1 kali kali 24 Sep 11 20:04 unique_syn_source_ports.txt

(kali@kali)~/SBT-DF203-Lab3
$ sha256sum -c reports/syn_capture_sha256.txt
evidence/mySYNFloodCapture.pcap: OK

(kali@kali)~/SBT-DF203-Lab3
$ sha256sum -c reports/normal_http_sha256.txt
evidence/normal_http.pcapng: OK

(kali@kali)~/SBT-DF203-Lab3
$ sha256sum -c reports/bounded_capture_hashes.txt
evidence/bounded_syn_activity.pcapng: OK
working/bounded_syn_activity_working.pcapng: OK

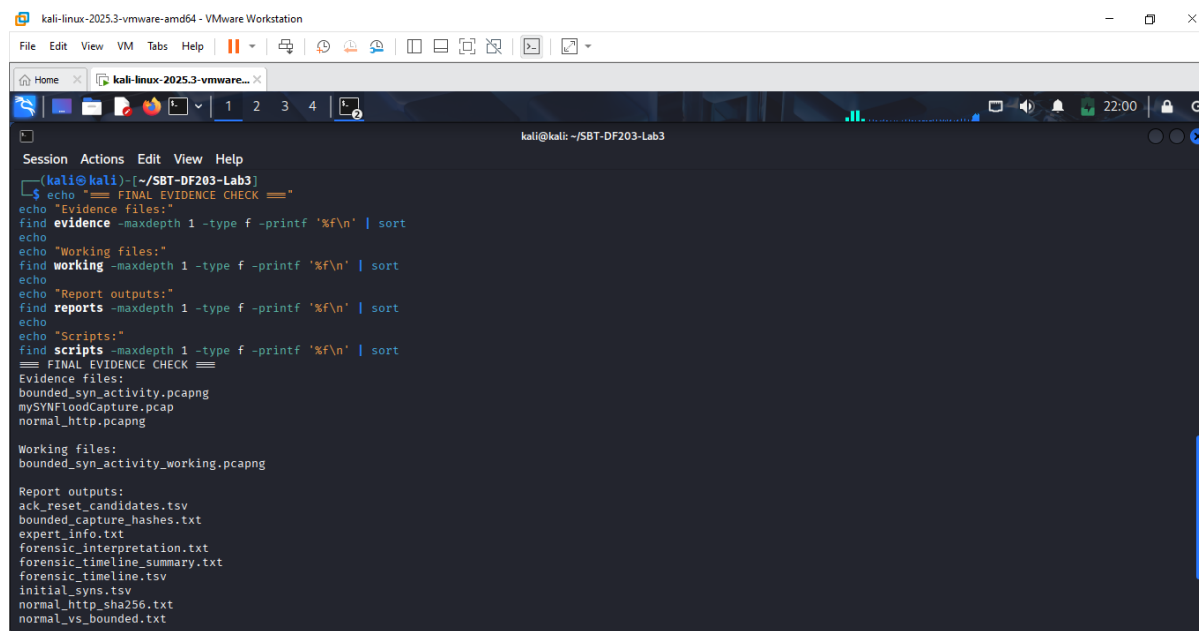
(kali@kali)~/SBT-DF203-Lab3
$ echo "=== SCRIPT ==="
ls -lh scripts/
=== SCRIPT ===
total 8.0K
drwxrwxr-x 2 kali kali 4.0K Sep 11 18:43 __pycache__
-rw-rw-r-- 1 kali kali 288 Sep 11 18:40 syn_probe_lab.py

(kali@kali)~/SBT-DF203-Lab3
$
```

Fig 4.7 Check all analysis outputs page 1.

Explanation

This screenshot shows the SHA-256 verification results returning OK for the evidence files. It also confirms that the bounded Scapy script used during the practical remained available in the scripts directory.



```
kali@kali: ~/SBT-DF203-Lab3
$ echo "=== FINAL EVIDENCE CHECK ==="
echo "Evidence files:"
find evidence -maxdepth 1 -type f -printf '%f\n' | sort
echo
echo "Working files:"
find working -maxdepth 1 -type f -printf '%f\n' | sort
echo
echo "Report outputs:"
find reports -maxdepth 1 -type f -printf '%f\n' | sort
echo
echo "Scripts:"
find scripts -maxdepth 1 -type f -printf '%f\n' | sort
=== FINAL EVIDENCE CHECK ===
Evidence files:
bounded_syn_activity.pcapng
mySYNFloodCapture.pcap
normal_http.pcapng

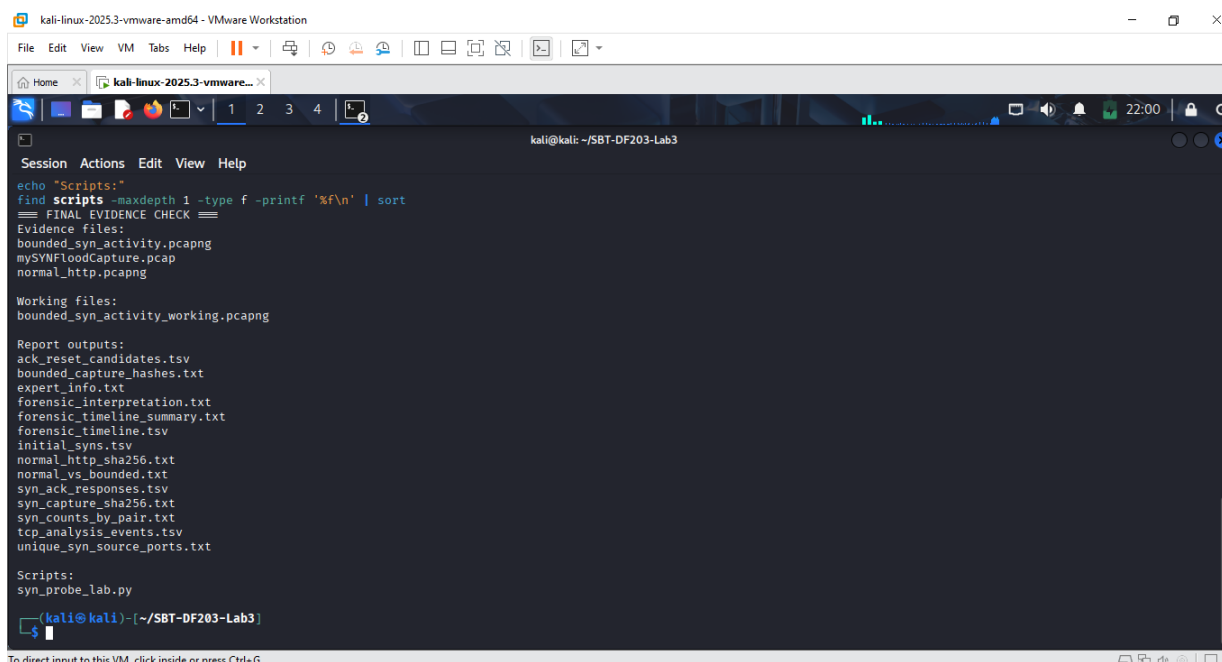
Working files:
bounded_syn_activity_working.pcapng

Report outputs:
ack_reset_candidates.tsv
bounded_capture_hashes.txt
expert_info.txt
forensic_interpretation.txt
forensic_timeline_summary.txt
forensic_timeline.tsv
initial_syns.tsv
normal_http_sha256.txt
normal_vs_bounded.txt
```

Fig 4.8 Check all analysis outputs page 2.

Explanation

This screenshot provides a final organised listing of the evidence, working and report files. It demonstrates that the required outputs were present before the report and submission package were finalised.



```
kali@kali: ~/SBT-DF203-Lab3
$ echo "Scripts:"
find scripts -maxdepth 1 -type f -printf '%f\n' | sort
=== FINAL EVIDENCE CHECK ===
Evidence files:
bounded_syn_activity.pcapng
mySYNFloodCapture.pcap
normal_http.pcapng

Working files:
bounded_syn_activity_working.pcapng

Report outputs:
ack_reset_candidates.tsv
bounded_capture_hashes.txt
expert_info.txt
forensic_interpretation.txt
forensic_timeline_summary.txt
forensic_timeline.tsv
initial_syns.tsv
normal_http_sha256.txt
normal_vs_bounded.txt
syn_ack_responses.tsv
syn_capture_sha256.txt
syn_counts_by_pair.txt
tcp_analysis_events.tsv
unique_syn_source_ports.txt

Scripts:
syn_probe_lab.py

(kali@kali)~[~/SBT-DF203-Lab3]
```

Fig 4.9 Check all analysis outputs page 3.

Explanation

This final screenshot confirms the remaining report outputs and the Scapy script in the completed lab directory. It serves as a final check that the practical evidence and generated analysis files were retained for submission.

NORMAL VS BOUNDED TRAFFIC COMPARISON

Indicator	Normal HTTP	Bounded SYN Activity
Initial SYNs	1	4
SYN-ACKs	1	4
Completed handshakes	1	0
Unique source ports	1	4
HTTP request	Yes	No
RST packets	0	4
Observed duration	0.026083665 s	0.004870879 s

The normal connection progressed from TCP establishment into HTTP communication. In contrast, every bounded connection stopped after the server's SYN-ACK and the following RST.

The comparison therefore provides a straightforward distinction between a successfully completed TCP session and the repeated incomplete or aborted sessions produced during the controlled simulation. Your existing report already records these measured values.

FORENSIC INTERPRETATION

The strongest finding from the investigation is the difference between the legitimate baseline and the bounded simulation.

The baseline completed:

SYN → SYN-ACK → ACK

and then progressed into HTTP communication.

The bounded traffic repeatedly produced:

SYN → SYN-ACK → RST

This confirms that the four connections did not complete normally.

However, this finding must be interpreted carefully. The capture was deliberately created using a four-packet training simulation and lasted only a few milliseconds. It therefore demonstrates an attack-like incomplete-handshake pattern but does not prove that Apache experienced a genuine SYN-flood denial of service.

There was no captured evidence showing sustained high-volume traffic, SYN-backlog exhaustion, prolonged service unavailability, increased response latency, resource exhaustion or failed legitimate connections.

A stronger conclusion would require packet-rate and duration measurements over a longer period together with system-state information and supporting Apache, firewall, operating-system or load-balancer logs. The manual specifically states that rate, duration, system impact and corroborating logs are needed for a strong denial-of-service conclusion.

LIMITATIONS

The main limitation of this exercise is that the simulated activity contained only four SYN packets. This was intentional and necessary to remain within the laboratory safety requirements, but it means the capture cannot demonstrate the scale or persistence normally associated with a real SYN flood.

The entire bounded sequence lasted approximately 0.004870879 seconds, which is too short to assess prolonged service degradation. No server CPU, memory, SYN-backlog or application-performance measurements were collected during the four-packet simulation.

The capture also came from the loopback interface, so the source and destination addresses were both 127.0.0.1. This differs from a real incident where traffic may arrive from one or more remote systems.

For these reasons, the findings should be limited to the packet behaviour actually observed and should not be presented as proof that a denial-of-service attack successfully affected the Apache service.

DETECTION AND MITIGATION RECOMMENDATIONS

The investigation shows that SYN traffic should not be assessed based only on the presence of SYN packets. Detection should instead look for sustained increases in the ratio of SYN packets to successfully completed handshakes.

Administrators should monitor SYN-backlog utilisation, connection-state tables, retransmission behaviour and service latency. During a real incident, packet evidence should also be correlated with Apache logs, firewall records, load-balancer information and operating-system logs so that suspicious network behaviour can be linked to any actual effect on the service.

Defensive measures may include enabling SYN cookies, using appropriate backlog and connection-timeout settings, applying upstream rate limiting and using DDoS-protection services where necessary. Packet or flow telemetry should also be retained, and system clocks should remain synchronised to support accurate forensic timelines. These are the controls recommended by the lab manual.

CONCLUSION

This practical successfully demonstrated how a normal TCP connection differs from repeated incomplete connection attempts.

The normal baseline showed a complete TCP three-way handshake followed by an HTTP GET request and an HTTP 200 OK response. The bounded Scapy test instead produced four initial SYN packets, four SYN-ACK responses and four RST packets, with zero completed handshakes and no HTTP request.

The investigation also demonstrated the importance of evidence preservation. SHA-256 hashes were recorded for the supplied capture, normal baseline and bounded simulation, while the identical hashes of the bounded original and working copy confirmed that the analysis was performed on an unchanged copy.

Although the bounded traffic reproduced an incomplete-handshake pattern associated with suspicious SYN activity, the evidence does not prove a real SYN-flood denial of service. The packet count, duration and absence of documented service impact are not sufficient for that conclusion. Additional rate, persistence and system-impact evidence would be required.

Screenshots

Fig 1.0 Lab folder structure.

Fig 1.1 Verifying Apache, TShark, Wireshark, Scapy.

Fig 1.2 Fig 1.1 Verifying Apache, TShark, Wireshark, Scapy page 1.

Fig 1.3 Starting Apache page and Confirming port 80.

Fig 1.4 Starting Apache page and Confirming port 80 page 1.

Fig 1.5 Downloading the PCAP and verifying the PCAP file.

Fig 1.6 Calculating the original SHA-256 hash and verifying the hash file was created.

Fig 1.7 Generating one legitimate HTTP request.

Fig 1.8 Generating one legitimate HTTP request page 1.

Fig 1.9 Showing the complete three-way handshake clearly.

Fig 2.0 Easier visual view.

Fig 2.1 Verifying the saved script.

Fig 2.2 Verifying the saved script.

Fig 2.3 Running and actively capturing on the loopback interface lo.

Fig 2.4 Sending the four authorized SYN packets.

Fig 2.5 Packets captured successfully.

Fig 2.6 Listing the bounded_syn_activity and details.

Fig 2.7 Preserving and hashing the both copies.

Fig 2.8 Hashing the normal baseline.

Fig 2.9 Initialing SYN Packets.

Fig 3.0 SYN ACK Responses.

Fig 3.1 Examining ACK and RST behaviour.

Fig 3.2 ACK RST Behaviour page 2.

Fig 3.3 Count SYNs by source/destination.

Fig 3.4 Extract unique SYN source ports.

Fig 3.5 Displaying both findings together.

Fig 3.6 TShark Expert Information.

Fig 3.7 TShark Expert TCP Analysis.

Fig 3.8 Verifying the normal baseline.

Fig 3.9 Verifying the bounded capture.

Fig 4.0 Comparison table.

Fig 4.1 Building the Forensic Timeline.

Fig 4.2 Building the Forensic Timeline page 1.

Fig 4.3 Forensic Timeline.

Fig 4.4 Forensic Timeline page 1.

Fig 4.5 Forensic Timeline page 2.

Fig 4.6 Check all analysis outputs.

Fig 4.7 Check all analysis outputs page 1.

Fig 4.8 Check all analysis outputs page 2.

Fig 4.9 Check all analysis outputs page 3.

References

ICDFA Materials.